



Universidad
Zaragoza

Trabajo Fin de Grado

Metadatos activos geoespaciales: estudio de
viabilidad con Datahub

Active Geospatial Metadata: Feasibility Study
with Datahub

Autor

Sergio Jiménez Frago

Director

Francisco Javier López Pellicer

ESCUELA DE INGENIERÍA Y ARQUITECTURA
2025

Agradecimientos

Para empezar, quiero expresar mi más profundo agradecimiento a mi tutor, Francisco Javier López Pellicer, por su ayuda, orientación y apoyo constante durante todo el desarrollo del proyecto. Su dedicación ha sido un pilar fundamental para la realización de este TFG.

También deseo agradecer al grupo de investigación al que pertenece, y en especial a Francisco Javier Zarazaga, por acogerme en su laboratorio durante este periodo y facilitarme un entorno de trabajo enriquecedor.

Asimismo, extiendo mi reconocimiento a todos los profesores del grado por transmitirme sus conocimientos y por su labor de enseñanza, gracias a la cual he podido aprender y crecer académicamente a lo largo de estos años.

Por último, quiero agradecer de corazón a todas las personas que me han acompañado en esta etapa: a mi familia y a mis amigos, por su apoyo incondicional, y muy especialmente a las personas que he conocido en el propio grado, que han hecho que esta experiencia sea no solo formativa, sino también divertida y humana. Gracias a ellos me llevo amistades que, sin duda, perdurarán en el tiempo.

Resumen

Los catálogos de metadatos geoespaciales actuales suelen basarse en modelos estáticos derivados de estándares como ISO 19115/19139 o DCAT. Aunque permiten publicar y buscar recursos, presentan limitaciones importantes: la información no puede actualizarse de forma granular, no existe sincronización con las fuentes de datos y la navegación entre recursos es limitada. Esto provoca que los catálogos ofrezcan metadatos incompletos o desactualizados, dificultando su reutilización y mantenimiento.

Este Trabajo Fin de Grado explora una alternativa moderna basada en metadatos activos mediante el uso de DataHub, una plataforma de gobernanza de metadatos construida sobre un modelo de grafo y un enfoque schema-first. El objetivo es evaluar si DataHub puede servir como base para un catálogo geoespacial dinámico capaz de representar y ampliar el modelo DCAT. Para ello, se desarrolla una prueba de concepto que incluye la definición de nuevas entidades y aspectos en DataHub, así como la creación de un proceso ETL que extrae metadatos ISO 19139 del catálogo IDEE y los transforma al modelo extendido.

Los resultados muestran que DataHub no solo reproduce con fidelidad la estructura de DCAT, sino que la mejora mediante propiedades dinámicas, relaciones navegables y consultas avanzadas basadas en GraphQL. Además, permite editar y actualizar metadatos sin necesidad de regenerar los ficheros originales, resolviendo uno de los principales problemas de los catálogos tradicionales. La comparación con GeoNetwork y el portal DCAT de la IDEE confirma que DataHub ofrece mayor flexibilidad, completitud y capacidad de exploración semántica.

En conjunto, el proyecto demuestra que DataHub es una plataforma viable para construir catálogos de metadatos geoespaciales dinámicos, aportando un enfoque más rico, extensible y adaptable que las soluciones tradicionales.

Índice

Lista de Figuras	v
Lista de tablas	vi
1 Introducción	1
1.1 Infraestructura de Datos Espaciales de España	1
1.2 Plataformas de Gobernanza de Metadatos	2
1.3 La gestión de datos geoespaciales abiertos.....	4
1.4 Motivación	5
1.5 Objetivos	5
1.6 Alcance	5
1.7 Metodología.....	6
2 Análisis	7
2.1 Vocabulario DCAT	7
2.1.1 Modelo base	8
2.1.2 Perfil de aplicación DCAT-AP	9
2.1.3 Perfil de aplicación GeoDCAT-AP	9
2.1.4 Elementos esenciales que deben conservarse	10
2.2 Los metadatos geográficos de la IDEE.....	11
2.3 Datahub: sistema y modelo de información	12
2.3.1 Arquitectura interna.....	16
2.3.2 Extensibilidad del modelo	17
2.4 Prueba de concepto	18
2.5 Requisitos.....	19
3 Diseño del sistema.....	20
3.1 Arquitectura del sistema final.....	20
3.2 Elementos principales	22
3.2.1 Entidades del modelo	22
3.2.2 Aspectos creados o extendidos	24
3.2.3 Propiedades clave incorporadas.....	24
3.2.4 Esquema resultante.....	24
3.3 Entorno de trabajo	25
3.4 Aspectos a destacar	26
4 Integración de DCAT en Datahub	27
4.1 Entidad Recurso	27
4.2 Entidad PlatformResource	28
4.3 Entidad CatalogRecord	29
4.4 Entidad Distribution.....	30
5 Pruebas de evaluación	32
5.1 Descripción de las pruebas.....	32
5.2 Actualización y mantenimiento de metadatos	33
5.3 Búsqueda y recuperación de información	34
5.4 Complejidad y riqueza de la información mostrada	36
5.5 Extensibilidad y escalabilidad del modelo	37
5.6 Resultados	38

6	Conclusiones.....	39
6.1	Objetivos conseguidos	39
6.2	Limitaciones	39
6.3	Mejoras	40
6.4	Conclusiones personales	41
	Bibliografía	42
	Anexo A Pasos para añadir una nueva entidad a Datahub	43
A.1	Añadir un nuevo tipo de entidad	43
A.2	Extender API de GraphQL.....	44
A.3	Extender el UI de React	49
	Anexo B Modelo DCAT completo	51
	Anexo C Tabla de requisitos	52

Lista de Figuras

Figura 1 - Portal Geonetwork de la IDEE.....	2
Figura 2 - Interfaz Web de DataHub.....	3
Figura 3 - DCAT Metadata Schema Simplificado	8
Figura 4 - Metadata Schema ISO19115	11
Figura 5 - Diagrama representativo del modelo de información de DataHub	13
Figura 6 - Diagrama de ejemplo de entidades de DataHub.....	15
Figura 7 - Grafo representativo de ejemplo de las entidades de Datahub	16
Figura 8 - Diagrama de componentes de Datahub.....	17
Figura 9 - Diagrama de arquitectura del sistema final	20
Figura 10 - Diagrama de clases resultante a la transformación.....	25
Figura 11 - Captura de pantalla de un Dataset en el portal de la IDEE	33
Figura 12 - Capturas de la visión y modificación de un Dataset en Datahub	34
Figura 13 - Captura de pantalla de la búsqueda en el portal de la IDEE	35
Figura 14 - Captura de pantalla de la búsqueda en Datahub	35
Figura 15 - Captura de pantalla de un Dataset de ejemplo con "Agua" en el título	36
Figura 16 - Captura de pantalla de un Dataset de ejemplo con el tag "Agua"	36
Figura 17 - Captura de pantalla de la adición de una Structured Property	37
Figura 18 - Diagrama de clases de DCAT completo	51

Lista de tablas

Tabla 1 - Entidades equivalentes DCAT-DataHub	22
Tabla 2 - Traducción de Dataset.....	28
Tabla 3 - Traducción de Resource	29
Tabla 4 - Traducción de CatalogRecord	30
Tabla 5 - Traducción de Distribution	31
Tabla 6 - Tabla de requisitos	52

1 Introducción

En este apartado se presenta el proyecto abordado. En primer lugar, se contextualiza el problema y los elementos principales que motivan su análisis. A continuación, se establecen los objetivos y el alcance del proyecto. Finalmente se describe de forma global la metodología utilizada.

1.1 Infraestructura de Datos Espaciales de España

La **Infraestructura de Datos Espaciales de España (IDEE)** [1] es el conjunto de tecnologías, políticas, normas y acuerdos institucionales orientados a facilitar el acceso, la interoperabilidad y el uso eficiente de la información geográfica en España. Su objetivo principal es integrar los datos espaciales producidos por las diferentes administraciones públicas, universidades y entidades privadas, garantizando su disponibilidad y reutilización mediante estándares abiertos.

La IDEE forma parte de la Infraestructura de Datos Espaciales de Europa (INSPIRE¹), iniciativa promovida por la Unión Europea para establecer una infraestructura común de información geográfica que permita la integración y comparación de datos entre países miembros.

Los metadatos que gestiona el nodo central de la IDEE describen los conjuntos de datos geográficos y los servicios relacionados con ellos de toda España, proporcionando información esencial sobre:

- La **procedencia** del dato (organismo responsable o fuente).
- La **temática** o **contenido** del conjunto de datos (por ejemplo, cartografía topográfica, catastro, medio ambiente, transporte, etc.).
- La **escala** y precisión de los datos.
- Las **condiciones de acceso y uso**, incluyendo licencias o restricciones.
- Los **formatos y sistemas de referencia espacial** utilizados.

Estos metadatos se ajustan a normas internacionales como **ISO 19115** [2] y a especificaciones INSPIRE, lo que permite su interoperabilidad con otras infraestructuras de datos espaciales a nivel europeo e internacional. Estos metadatos son metadatos estáticos porque no se actualizan automáticamente y deben regenerarse cada vez que la información cambia.

En la práctica, la IDEE se materializa en un portal web² que actúa como punto de acceso a servicios de visualización como, por ejemplo, Web Map Service, de descarga como, por ejemplo, Web Feature Service y Web Coverage Service,

¹ <https://knowledge-base.inspire.ec.europa.eu/>

² <https://www.idee.es>

catálogos de metadatos como, por ejemplo, Catalog Service for the Web (CSW) y otros servicios de geoprocésamiento como, por ejemplo, Web Processing Service, entre otros, facilitando así el intercambio y aprovechamiento de información geográfica entre organismos y usuarios. De hecho, el servicio CSW será el más útil y el que más veremos en este proyecto, ya que la IDEE expone un endpoint CSW³ gestionado mediante GeoNetwork, proporcionando acceso directo a los metadatos publicados.

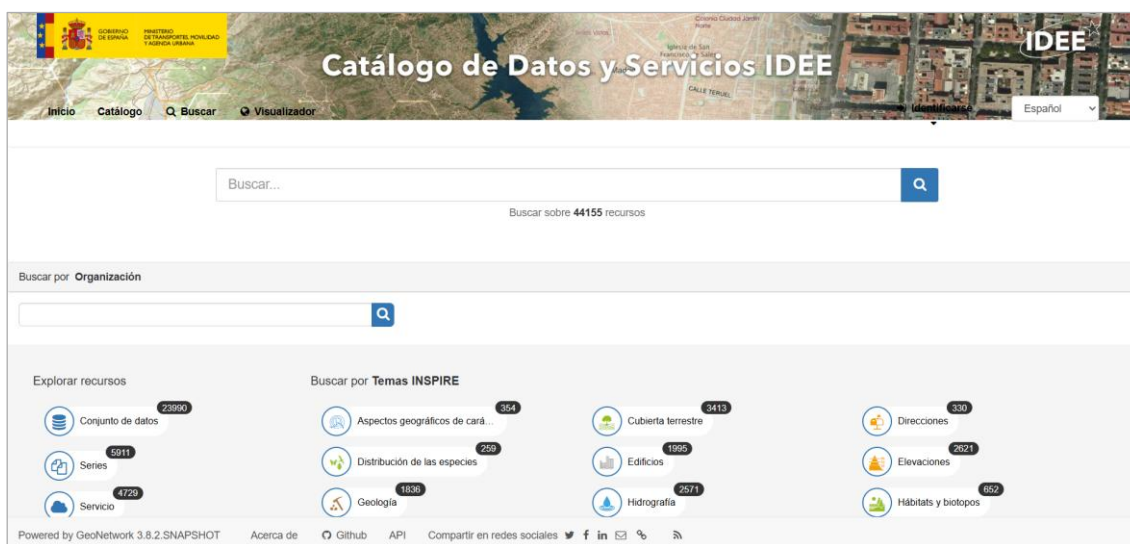


Figura 1 - Portal Geonetwork de la IDEE

1.2 Plataformas de Gobernanza de Metadatos

Una **plataforma de gobernanza de metadatos** es una plataforma o sistema centralizado que permite la integración, gestión y distribución de datos procedentes de múltiples fuentes, con el objetivo de facilitar su descubrimiento, gobierno y reutilización dentro de una organización o comunidad. A diferencia de los sistemas tradicionales de almacenamiento (como los data warehouses o data lakes), una plataforma de gobernanza de metadatos no se centra únicamente en la acumulación de datos, sino en la gestión de metadatos y la conectividad entre sistemas. Es decir, a diferencia de los catálogos tradicionales, estos metadatos son considerados **metadatos activos** a diferencia de los metadatos estáticos que he presentado en la sección anterior. Gracias a los metadatos activos, su función principal es actuar como un catálogo de datos que describe, organiza y hace accesible la información disponible, independientemente de dónde se almacene realmente. Estos permiten superar las limitaciones de los metadatos estáticos ya que permiten una gestión dinámica y automatizada de la información, facilitando la actualización y sincronización continua con las fuentes de datos.

³ <https://www.idee.es/csw-inspire-idee/srv/>

Entre los principales elementos que maneja una plataforma de gobernanza de metadatos destacan:

- **Metadatos técnicos:** estructura, formato, esquemas o tipología de los datos.
- **Metadatos de negocio:** significado, propietario, políticas de uso y clasificación.
- **Metadatos operacionales:** frecuencia de actualización, calidad y procedencia de los datos.
- **Linaje de datos (data lineage):** trazabilidad del origen y transformación de cada conjunto de datos.

Las plataformas de gobernanza de metadatos suelen incorporar capacidades de búsqueda semántica, versionado de metadatos, control de acceso basado en roles y API de interoperabilidad, lo que los convierte en herramientas clave para la **gobernanza de datos** (data governance) en entornos empresariales o institucionales.

Ejemplos conocidos de estas plataformas son DataHub⁴ [3] (ver Figura 2), OpenMetadata⁵, Amundsen⁶ (de Lyft) o soluciones comerciales como Collibra⁷ y Informatica Data Catalog⁸. Estas plataformas permiten crear un ecosistema de datos unificado, transparente y colaborativo, facilitando la toma de decisiones basada en información confiable y documentada.

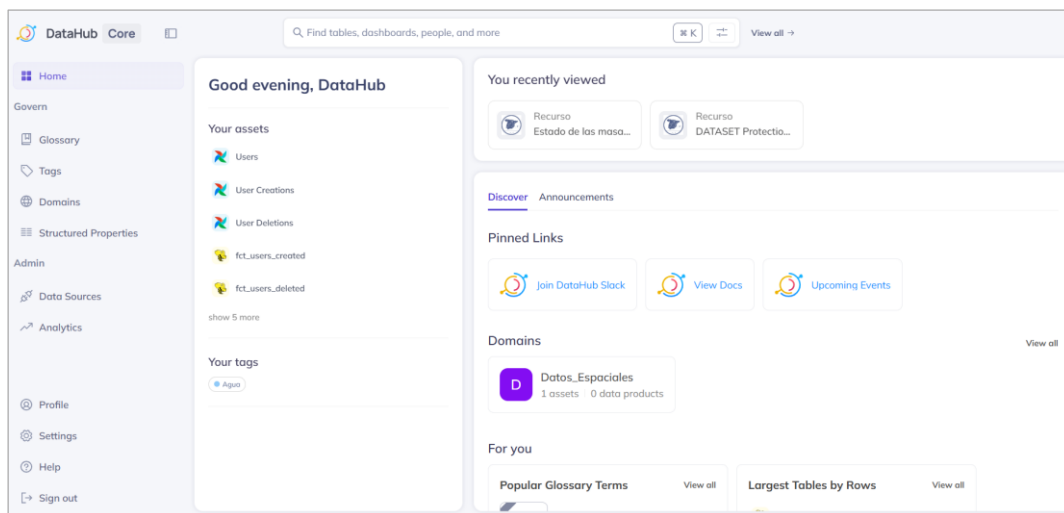


Figura 2 - Interfaz Web de DataHub

⁴ <https://docs.datahub.com/docs/features>

⁵ <https://open-metadata.org/>

⁶ <https://www.amundsen.io/>

⁷ <https://www.collibra.com/>

⁸ <https://www.informatica.com/es/products/data-catalog.html>

En particular, DataHub destaca por su arquitectura basada en aspectos, su fuerte orientación a grafos y su capacidad de extender el modelo sin modificar el núcleo del sistema, lo que lo convierte en una solución especialmente flexible y adaptable para catálogos complejo.

1.3 La gestión de datos geoespaciales abiertos

En el ámbito de la gestión de datos geoespaciales abiertos, los catálogos de metadatos juegan un papel fundamental para garantizar la interoperabilidad y accesibilidad de la información. Tradicionalmente, estándares internacionales como ISO 19115 [2], se utilizan para describir metadatos en estos catálogos, proporcionando un marco común para la publicación y búsqueda de recursos geoespaciales, y su codificación se realiza en XML acorde con el estándar ISO 19139.

Actualmente, la Unión Europea impulsa además el uso del Data Catalog Vocabulary (DCAT) [4] como vocabulario común para la publicación de datos abiertos. DCAT es un vocabulario RDF diseñado para describir catálogos y conjuntos de datos y constituye la base para las descripciones de datos abiertos del sector público europeo, tras su adopción por el programa ISA de la Comisión Europea. En este contexto, muchos catálogos nacionales —como el de datos.gob.es— publican sus metadatos en formato DCAT, aunque la información original proceda de modelos tradicionales como ISO 19115, tal y como ocurre en la propia IDEE, que genera sus metadatos en ISO 19139 pero los expone también en DCAT para facilitar su interoperabilidad.

Sin embargo, estos catálogos suelen emplear metadatos estáticos, que resultan difíciles de mantener actualizados debido a la evolución constante de los datos y la información asociada. Este problema de mantenimiento limita la capacidad de los usuarios y sistemas para acceder a información precisa y actualizada.

En este contexto, usar una plataforma de gobernanza de metadatos que ofrece capacidades para gestionar metadatos procedentes de catálogos geoespaciales. Este proyecto se centra en explorar la viabilidad del uso de una de las plataformas de gobernanza de metadatos mencionada anteriormente, DataHub, para implementar un catálogo activo mediante una prueba de concepto basada en los metadatos del catálogo de la IDEE, comparándolo con soluciones tradicionales como GeoNetwork [5].

1.4 Motivación

La principal motivación de este trabajo es la necesidad de mejorar la gestión y mantenimiento de catálogos geoespaciales mediante metadatos activos, reduciendo el esfuerzo manual asociado a la actualización de metadatos estáticos y aumentando la fiabilidad de la información ofrecida a los usuarios finales. El uso de metadatos activos tiene el potencial de automatizar procesos, mejorar la calidad de los datos y facilitar la interoperabilidad entre sistemas. Además, el análisis de plataformas emergentes como DataHub, frente a soluciones tradicionales como GeoNetwork, permitirá identificar ventajas y limitaciones de estas, contribuyendo a la mejora continua de los sistemas de gestión de datos geoespaciales abiertos.

1.5 Objetivos

Este Trabajo Fin de Grado aborda el estudio de viabilidad del uso de DataHub como plataforma para metadatos activos geoespaciales, a través de los siguientes objetivos específicos:

- Realizar una revisión de los estándares y soluciones actuales en catálogos geoespaciales, con especial atención a la gestión de metadatos estáticos y activos.
- Desarrollar una prueba de concepto basada en DataHub, que permita la integración y gestión dinámica de metadatos extraídos del catálogo de la IDEE.
- Comparar DataHub con otras soluciones existentes, como GeoNetwork, en términos de funcionalidad, flexibilidad y facilidad de mantenimiento.
- Evaluar la viabilidad técnica y práctica del uso de DataHub en entornos reales de catálogos geoespaciales.
- Proponer mejoras y recomendaciones para la adopción de metadatos activos en la gestión de datos geoespaciales abiertos.

1.6 Alcance

El alcance de este proyecto incluye la implementación y evaluación de una prueba de concepto que utiliza DataHub para gestionar metadatos activos, desplegada en un entorno controlado mediante contenedores Docker sobre una máquina virtual Ubuntu. Se han modificado y desarrollado componentes en Java para la extracción y transformación de datos, así como scripts de automatización y pruebas automatizadas en Python con pytest para verificar la integridad y funcionamiento del sistema.

Los resultados obtenidos permiten valorar la aplicabilidad de DataHub en la gestión de metadatos geoespaciales activos y compararla con soluciones tradicionales, sin abordar en detalle la integración completa en entornos productivos ni la escalabilidad a gran escala, que podrían ser objeto de trabajos futuros.

1.7 Metodología

El proyecto siguió una metodología iterativa basada en la experimentación y mejora continua. Primero se desplegó una instancia de DataHub en un entorno controlado mediante Docker y WSL, y posteriormente se desarrollaron y modificaron componentes en Java usando IntelliJ IDEA para gestionar los metadatos de la IDEE. En paralelo, se implementaron pruebas automatizadas con pytest para validar la extracción, transformación y carga de metadatos. También se realizó una comparación práctica y bibliográfica entre DataHub y GeoNetwork. El trabajo se organizó en fases: creación de una prueba de concepto, integración con metadatos reales, mejoras funcionales y validación, y evaluación final. Este enfoque permitió comprobar la viabilidad y ventajas de utilizar metadatos activos en entornos geoespaciales.

2 Análisis

El propósito de esta fase es estudiar en detalle los modelos y estándares implicados en el proyecto con el fin de establecer una base sólida para su posterior adaptación e implementación en DataHub. Para ello, se analizan el vocabulario DCAT y sus extensiones, los metadatos geoespaciales de la IDEE basados en ISO 19115/19139 y la arquitectura interna y modelo de información de DataHub. Este análisis permitirá identificar equivalencias, detectar incompatibilidades y definir los elementos esenciales que deberán trasladarse sin pérdida de información, garantizando la coherencia semántica y la escalabilidad del sistema final.

2.1 Vocabulario DCAT

El **Data Catalog Vocabulary (DCAT)** [4] es un vocabulario estandarizado desarrollado para mejorar la interoperabilidad entre catálogos de datos publicados en la web. Su origen se remonta a los trabajos iniciales realizados en DERI, que posteriormente fueron ampliados por el grupo de interés eGov del W3C [6] y consolidados dentro del proceso de estandarización del grupo de trabajo *Government Linked Data*. Hoy en día, DCAT constituye la base para las descripciones de conjuntos de datos abiertos en la administración pública europea y ha sido adoptado y extendido por el programa ISA de la Comisión Europea. Al estar construido sobre RDF, permite integrar los metadatos en el ecosistema de la Web Semántica y enlazarlos con otros vocabularios, recursos o grafos de conocimiento. Además, su uso resulta necesario porque Dublin Core y DCTerms, aunque útiles para metadatos básicos, no proporcionan la estructura ni las entidades específicas requeridas para describir catálogos, distribuciones y relaciones complejas entre recursos.

Dublin Core Terms (DCTerms) es un conjunto ampliado de términos del estándar Dublin Core que proporciona propiedades más específicas y precisas para la descripción de recursos. Mientras que el núcleo de Dublin Core contiene solo 15 elementos básicos, DCTerms añade definiciones más detalladas, relaciones semánticas y especificaciones de rango y dominio, lo que permite describir recursos con mayor exactitud dentro de sistemas basados en metadatos y en la Web Semántica.

Para este trabajo, el interés principal de DCAT radica en identificar qué entidades y propiedades son esenciales para poder trasladar posteriormente este modelo a DataHub con la menor pérdida de información.

2.1.1 Modelo base

El esquema de metadatos de DCAT que se pretende replicar es el que se describe en la Figura 3 DCAT se organiza en torno a un conjunto de clases fundamentales que estructuran todo el catálogo:

- **dcat:Resource**: clase abstracta y general que actúa como punto común del resto de elementos del modelo. Sirve como superclase de Dataset, DataService y Distribution, proporcionando un marco unificado para describir cualquier tipo de recurso dentro del catálogo. No suele emplearse directamente en los metadatos, pero establece propiedades genéricas como el título, la descripción, las relaciones con otros recursos o enlaces asociados. Su función es ofrecer una base semántica común que permita tratar datasets, servicios y distribuciones como recursos interoperables dentro del ecosistema DCAT.
- **dcat:Dataset**: clase central del modelo. Representa el recurso informativo y concentra la mayor parte de las propiedades semánticas relevantes: identificadores, descripciones, temas, palabras clave, productores, fechas, extensión espacial/temporal, etc.
- **dcat:Catalog**: clase que agrupa todos los recursos publicados. No contiene datos por sí misma, sino enlaces a datasets y servicios.
- **dcat:DataService** (incorporado en DCAT 3): permite representar servicios de acceso a datos, algo habitual en IDE (WMS, WFS, API REST).
- **dcat:Distribution**: describe cómo está disponible el dataset (formato, URL de descarga, tipo de servicio, mediaType, etc.).
- **dcat:CatalogRecord**: metadatos sobre la entrada de catálogo (no sobre el dataset en sí). Permite describir cuándo se creó, modificó o importó el registro.

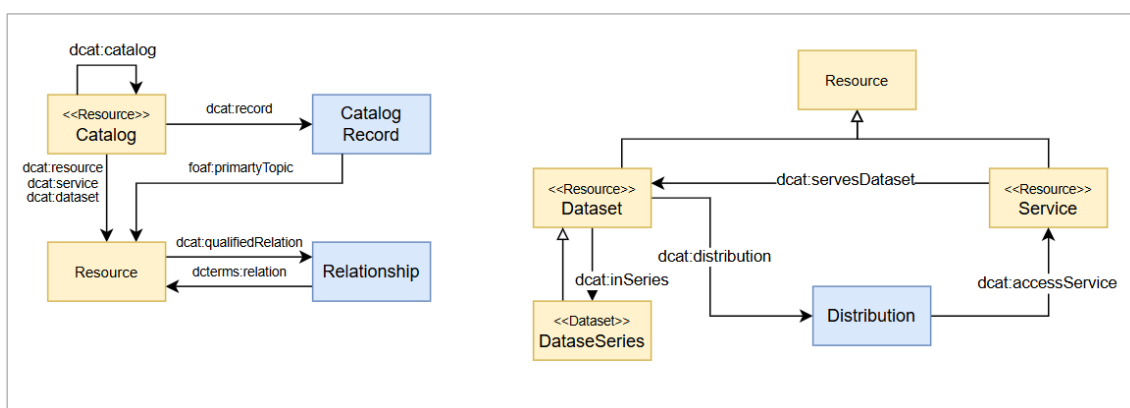


Figura 3 - DCAT Metadata Schema Simplificado

El análisis de estas clases permite identificar la función semántica de cada pieza del modelo. DCAT no proporciona únicamente campos descriptivos, sino una separación lógica entre recurso (Dataset), sus representaciones (Distribution), su procedencia (CatalogRecord) y los servicios asociados (DataService). Esta separación es clave para trasladar el modelo a DataHub manteniendo la coherencia.

2.1.2 Perfil de aplicación DCAT-AP

El perfil de **aplicación DCAT Application Profile for Data Portals in Europe (DCAT-AP)** [7] es una extensión del vocabulario DCAT promovida por la Comisión Europea y el programa ISA². Su objetivo es unificar la forma en la que los portales de datos europeos describen y publican su información, facilitando la interoperabilidad entre los catálogos nacionales y el portal europeo de datos. DCAT-AP no redefine el modelo, sino que introduce **restricciones y ampliaciones** destinadas a garantizar interoperabilidad entre portales de datos europeos.

Desde un punto de vista analítico, DCAT-AP aporta:

- **Obligatoriedad de ciertos campos** (dct:title, dct:description, dct:publisher, dct:identifier, etc.).
- **Uso de vocabularios controlados**, por ejemplo, para licencias, categorías temáticas, formatos o tipos de recurso.
- **Estructuración explícita de agentes** (foaf:Agent) para representar organizaciones públicas.
- **Clarificación de cardinalidades y relaciones**, reduciendo la ambigüedad del modelo original de DCAT.

En resumen, DCAT-AP introduce normativa semántica, no nuevas entidades, y obliga a que los metadatos sean completos y coherentes. Esa consistencia debe mantenerse al trasladar la información a DataHub.

2.1.3 Perfil de aplicación GeoDCAT-AP

GeoDCAT-AP [8] es una extensión geoespacial de DCAT-AP diseñada para representar metadatos provenientes de estándares como ISO 19115 e INSPIRE metadata guidelines⁹, de forma que puedan integrarse dentro de catálogos basados en DCAT. GeoDCAT-AP amplía DCAT-AP mediante mapeos formales entre los elementos de ISO 19115 (estándar de metadatos geográficos que se describirá en la sección 2.2) y las clases de DCAT.

⁹ https://knowledge-base.inspire.ec.europa.eu/metadata-technical-guidelines_en

El análisis muestra que GeoDCAT-AP aporta:

- **Propiedades geoespaciales** (bounding box, resolución espacial, sistema de referencia, escala).
- **Propiedades temporales detalladas** (periodos de validez, series temporales).
- **Propiedades INSPIRE** relacionadas con conformidad, el grado de cumplimiento y categorías temáticas.
- **Mapeos uno-a-uno entre elementos ISO y propiedades DCAT / DCTERMS.**

Estas extensiones permiten que metadatos geográficos puedan representarse en DCAT sin pérdida semántica.

Dado que la IDEE publica sus metadatos ISO 19115 codificados en ISO 19139, GeoDCAT-AP actúa como puente conceptual necesario para trasladar esta estructura hacia un sistema como DataHub.

2.1.4 Elementos esenciales que deben conservarse

A partir del análisis general del vocabulario DCAT y sus perfiles, pueden identificarse los elementos imprescindibles que deben preservarse en cualquier proceso de adaptación o traslado del modelo:

- **La estructura jerárquica del catálogo**, basada en las clases Catalog, Dataset, Distribution, DataService y CatalogRecord.
- **La separación conceptual entre los recursos**, sus representaciones y su procedencia:
 - Dataset → recurso principal.
 - Distribution → forma de acceso al recurso.
 - CatalogRecord → información sobre la entrada en el catálogo.
- **Los metadatos obligatorios y vocabularios controlados definidos por DCAT-AP**, que garantizan coherencia y completitud.
- **Los elementos geoespaciales definidos por GeoDCAT-AP**, especialmente relevantes en contextos IDE.
- **Las relaciones entre clases**, que permiten mantener la conexión entre datasets, distribuciones, servicios y catálogos.
- **Los atributos clave que aparecen en varias clases** (como identificadores, descripciones, agentes responsables o fechas), cuya consistencia semántica es fundamental para cualquier transformación.

Este estudio permitirá adaptar y trasladar el esquema de DCAT a nuestro sistema de manera correcta, garantizando la importación de los datos sin pérdida de información, manteniendo la consistencia semántica y asegurando que la solución sea escalable y fácilmente extensible en el futuro.

2.2 Los metadatos geográficos de la IDEE

La **Infraestructura de Datos Espaciales de España (IDEE)** [1] publica sus metadatos a través de servicios interoperables siguiendo los estándares definidos por la Directiva INSPIRE. Estos metadatos están disponibles en distintos formatos, siendo los principales DCAT —empleado para la publicación en portales de datos abiertos— e ISO 19115/19139. En este caso, ISO 19115 se refiere al modelo de información para metadatos geoespaciales, mientras que ISO 19139 define la implementación y codificación en XML de ese modelo (Figura 4)

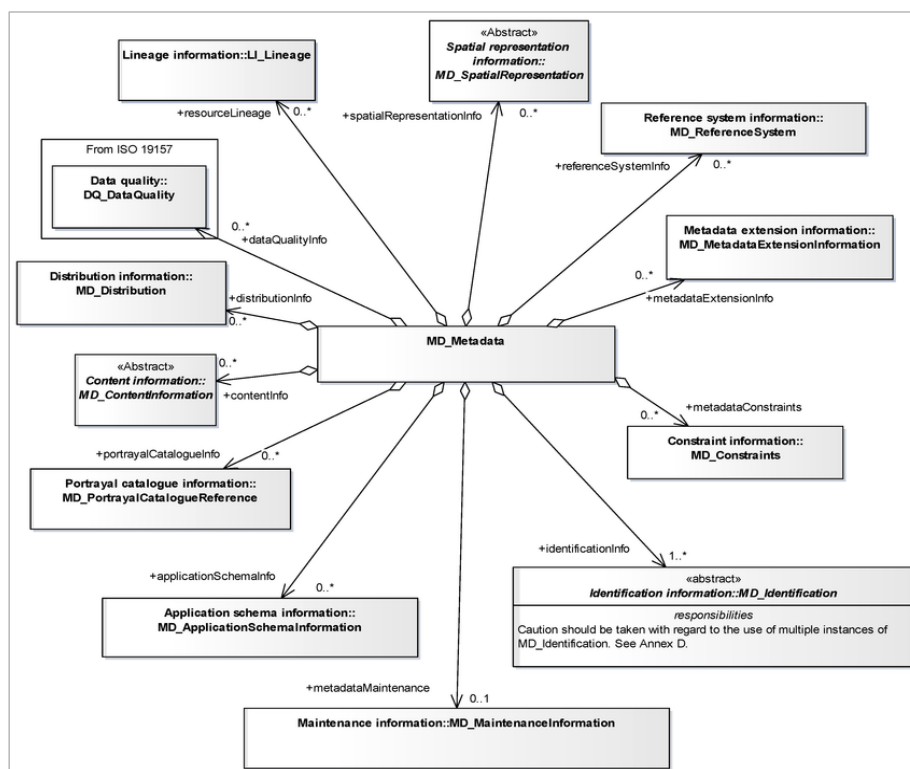


Figura 4 - Metadata Schema ISO19115

Aunque la IDEE ofrece un catálogo DCAT accesible desde su portal y utilizable para consultas generales, este formato tiene un nivel de detalle limitado en comparación con las especificaciones ISO. DCAT está orientado principalmente a la interoperabilidad web y a la publicación de recursos de forma sencilla, mientras que los modelos ISO 19115 y su codificación XML ISO 19139 proporcionan una descripción exhaustiva y estructurada del recurso geográfico, incluyendo metadatos técnicos, geoespaciales y de calidad que no siempre están presentes o completamente representados en DCAT.

En el caso concreto de este estudio, resulta más adecuado partir de los metadatos ISO 19115 codificados en XML ISO 19139 por varias razones:

- **Mayor nivel de detalle:** ISO 19115 describe información geográfica compleja, como sistemas de referencia, extensión espacial (bounding box), resoluciones espaciales, calidad del dato o linaje, elementos fundamentales en el contexto de las IDE. DCAT, incluso con GeoDCAT-AP, ofrece una representación más abstracta y en algunos casos simplificada.
- **Estructura fuertemente tipada:** la codificación XML ISO 19139 mantiene la jerarquía interna del metadato y garantiza la existencia de elementos obligatorios definidos por INSPIRE. Esto facilita identificar los componentes esenciales que deben conservarse en cualquier proceso de transformación.
- **Consistencia semántica:** el estándar ISO es el modelo fuente utilizado por la IDEE; por tanto, trabajar directamente sobre sus metadatos evita depender de conversiones previas o posibles pérdidas de información en el proceso de publicación a DCAT.
- **Correspondencia formal con GeoDCAT-AP:** dado que GeoDCAT-AP proporciona mapeos explícitos entre ISO 19115/19139 y DCAT, analizar primero la estructura ISO permite comprender con precisión qué elementos deben preservarse antes de representar los recursos en un modelo más generalista.

2.3 Datahub: sistema y modelo de información

DataHub es una plataforma de catalogación de metadatos diseñada con un enfoque fuertemente modular y extensible, adecuada para gestionar catálogos heterogéneos y en evolución constante. Su arquitectura parte de un modelo *schema-first*, definido mediante el lenguaje Pegasus PDL¹⁰, donde el núcleo conceptual se articula alrededor de tres elementos fundamentales: entidades, aspectos y propiedades. Esta separación estructural permite representar la información como un grafo de metadatos flexible, versionable y fuertemente tipado. En la Figura 5 podemos ver un simple esquema del modelo de información de DataHub.

¹⁰ https://linkedin.github.io/rest.li/pdl_schema

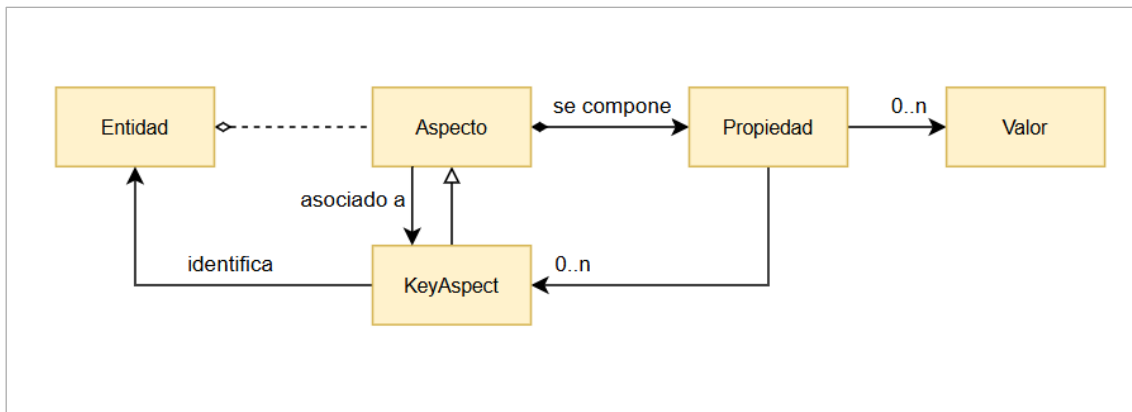


Figura 5 - Diagrama representativo del modelo de información de DataHub

Entidades. Las entidades son los nodos principales del grafo de DataHub. Representan objetos de metadatos como *Dataset*, *Chart*, *Dashboard*, *DataPlatform* o *CorpUser*. Cada entidad se caracteriza por:

- un **URN** que actúa como identificador único;
- un **aspecto clave (key aspect)** que define los atributos mínimos necesarios para identificarla;
- un conjunto dinámico de **aspectos adicionales**, que contienen la información real asociada a la entidad y que se asocian a la entidad asociándolos al aspecto clave.

Las entidades, por tanto, no almacenan información de forma directa: su contenido emerge de los aspectos que tienen asociados. Esto las convierte en estructuras esencialmente “virtuales”, cuya definición depende de los componentes que las completan.

Aspectos. Los aspectos constituyen la unidad mínima de información en DataHub y también la unidad mínima de escritura. Cada aspecto:

- encapsula un bloque coherente de metadatos (por ejemplo, *DatasetProperties*, *Ownership*, *Tags*, *InstitutionalMemory*);
- puede versionarse y actualizarse de manera independiente del resto de aspectos de la entidad;
- puede reutilizarse entre múltiples entidades, proporcionando consistencia semántica en todo el catálogo.

Además, los aspectos pueden incluir **relaciones** con otras entidades mediante campos anotados con `@Relationship`, que generan aristas que se pueden recorrer en ambos sentidos dentro del grafo de metadatos.

Propiedades Estructuradas. Dentro de los aspectos, DataHub permite definir propiedades tradicionales derivadas del esquema PDL y también **structured properties** (aspecto `StructuredProperties`), que actúan como extensiones dinámicas. Estas últimas permiten añadir nuevos campos sin crear un aspecto distinto, manteniendo así la flexibilidad del modelo sin comprometer su estructura lógica.

Identificadores: Key y URN. Los aspectos clave, o key aspects, como *DatasetKey*, *ChartKey* o *DashboardKey*, contienen los campos que identifican de manera única a una entidad. Estos aspectos se serializan como URN, lo que permite:

- identificar una entidad sin necesidad de materializarla completamente,
- navegar el grafo mediante identificadores compactos,
- consultar entidades directamente desde la API o GraphQL.

Los *key aspects* pueden considerarse “aspectos virtuales”, ya que su función es únicamente definitoria y no contienen información funcional adicional.

Relación entre entidades y aspectos: similitud con ISO 19115. El modelo de DataHub presenta una notable similitud estructural con el modelo conceptual de ISO 19115 (Figura 4). En ambos casos existe una entidad principal —*MD_Metadata* en ISO y la entidad base en DataHub— y un conjunto de componentes auxiliares que encapsulan la información real (clases asociadas en ISO, aspectos en DataHub). Este paralelismo refuerza que DataHub pueda adaptarse con facilidad a modelos complejos como DCAT o GeoDCAT-AP.

Esto se observa claramente en el ejemplo de la Figura 6. Las entidades *Chart* y *Dashboard* solo están definidas a partir de sus aspectos clave (*ChartKey* y *DashboardKey*). Toda la información descriptiva se distribuye en aspectos como *ChartInfo*, *DashboardInfo* u *Ownership*. Además, un aspecto como *Ownership* puede asociarse a múltiples entidades sin necesidad de duplicarlo, aprovechando así el carácter reusable del modelo.

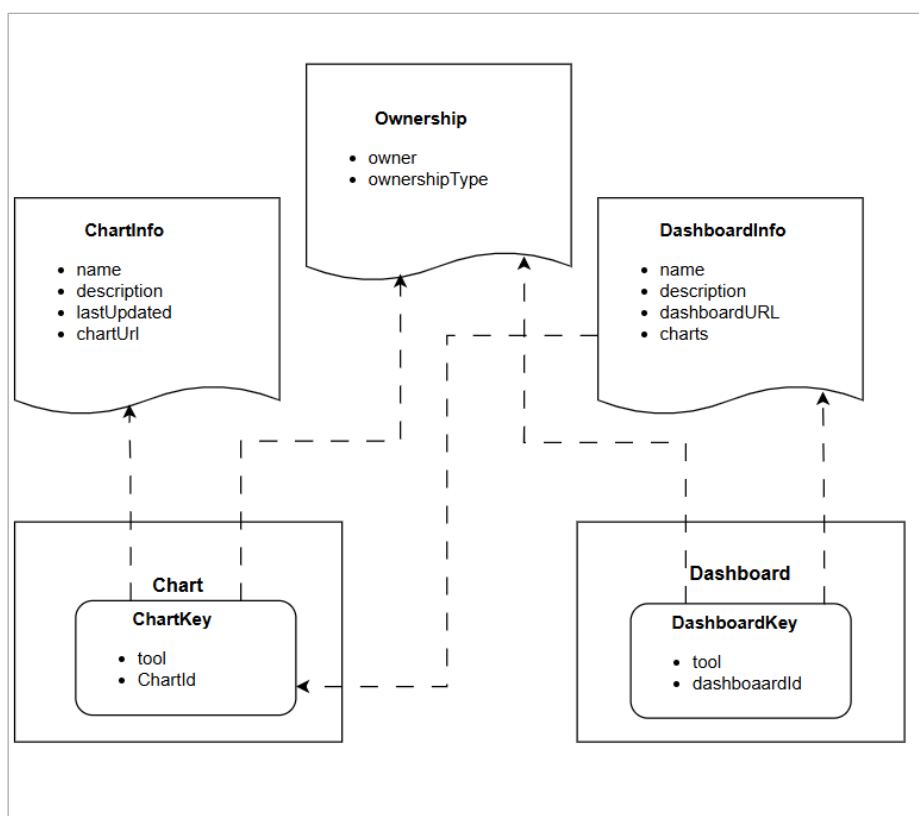


Figura 6 - Diagrama de ejemplo de entidades de DataHub

Debido a que la información se organiza en aspectos que pueden referenciar a otros mediante relaciones explícitas, el modelo interno de DataHub se comporta como un grafo de nodos y aristas. La Figura 7 lo muestra de forma esquemática: los nodos verdes representan los aspectos que definen las entidades, mientras que el resto de los nodos constituyen propiedades o componentes auxiliares enlazados entre sí.

Relación entre DataHub y grafos. Además, esta estructura justifica el uso de GraphQL como mecanismo de acceso y consulta. Al tratarse de un grafo de entidades y aspectos, GraphQL permite recuperar únicamente los nodos y relaciones necesarios, construir consultas que siguen las conexiones entre aspectos y obtener una vista unificada de la información dispersa en el modelo. Esto facilita la exploración del grafo de metadatos y permite optimizar el intercambio de información entre el cliente y el servidor.

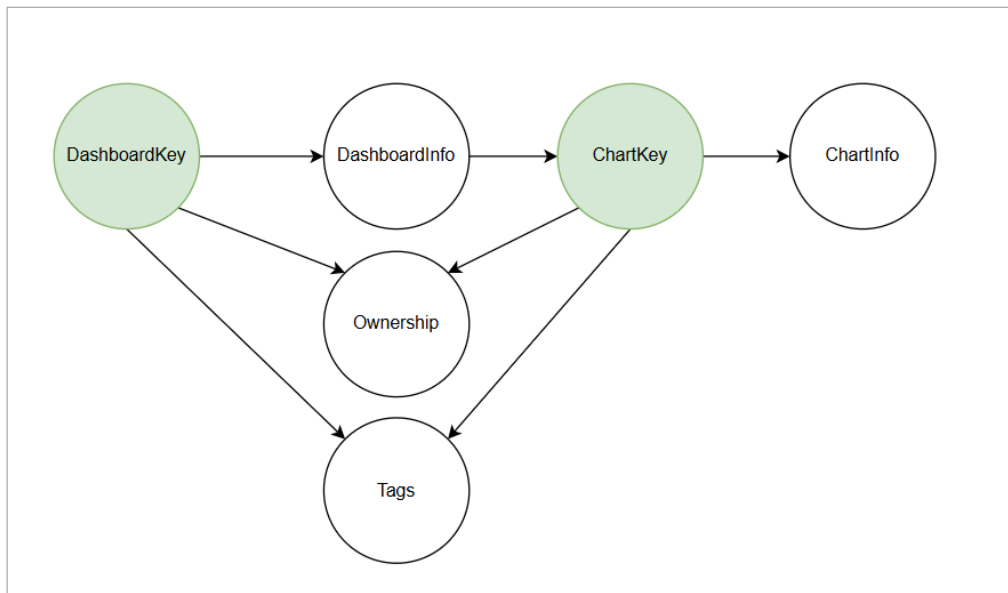


Figura 7 - Grafo representativo de ejemplo de las entidades de Datahub

2.3.1 Arquitectura interna

DataHub está compuesto por varios módulos principales, cada uno con responsabilidades específicas dentro del sistema:

- **Metadata Models (Pegasus PDL):** Define el esquema de metadatos mediante archivos PDL. De estos PDL se generan automáticamente clases Java y el esquema GraphQL.
- **GMS (Generalized Metadata Service):** Servicio backend en Java responsable de almacenar y versionar los *aspectos*, mantener la integridad del grafo de metadatos, resolver relaciones entre entidades, validar los esquemas PDL, y coordinar la propagación de cambios hacia los distintos servicios del sistema.
- **GraphQL API:** Ofrece el punto de acceso principal para consultar entidades y navegar por sus relaciones.
- **UI Web (React):** Interfaz de usuario que permite explorar entidades, visualizar aspectos y administrar el catálogo.
- **Ingestion Framework (Python):** Sistema para conectar DataHub con fuentes externas e importar metadatos.
- **Infraestructura base.** DataHub usa Kafka para mensajería y notificaciones de cambios, OpenSearch / Elasticsearch para indexación y búsqueda, y MySQL / PostgreSQL / SQLite para persistencia de aspectos.

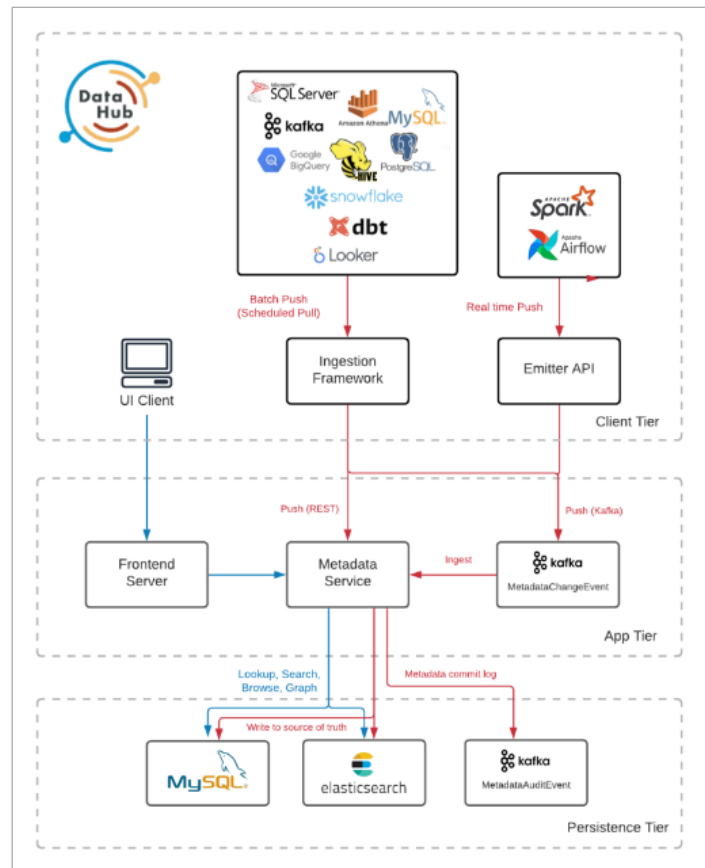


Figura 8 - Diagrama de componentes de Datahub¹¹

Esta arquitectura modular y orientada a flujos de metadatos permite que DataHub escale, integre nuevas fuentes y extienda su modelo sin modificar su núcleo.

2.3.2 Extensibilidad del modelo

Uno de los elementos más relevantes del análisis es la capacidad de DataHub para extender su modelo de información:

- Se pueden crear nuevas entidades definiendo nuevos aspectos clave.
- Se pueden añadir nuevos aspectos a entidades existentes.
- Se puede extender el esquema GraphQL para exponer nuevos campos y entidades.
- Se puede ampliar la interfaz añadiendo componentes para visualizar los nuevos metadatos.
- El aspecto *StructuredProperty* permite extender el modelo sin modificar el código base.

¹¹<https://github.com/datahub-project/static-assets/blob/main/imgs/saas/DataHub-Architecture.png> Copyright 2015 LinkedIn Corp. All rights reserved.

En conjunto, este diseño modular y escalable facilita la representación de modelos de metadatos heterogéneos y evolutivos, permitiendo adaptar la plataforma a dominios especializados sin comprometer su arquitectura interna. Esto enfoca el proyecto hacia una metodología iterativa en la que el modelo se vaya extendiendo progresivamente.

2.4 Prueba de concepto

La primera fase del proyecto consistió en una prueba de concepto con DataHub para evaluar su viabilidad y flexibilidad a la hora de adaptar el modelo de metadatos a los catálogos propios. El objetivo principal fue comprobar si era posible extender el sistema para incorporar nuevas entidades y atributos específicos de nuestros catálogos de datos.

Se inició con la instalación y despliegue local de DataHub mediante Docker, lo que permitió familiarizarse con la arquitectura, la interfaz gráfica y la línea de comandos. Posteriormente, se configuró un entorno de desarrollo local con Java, Node.js y un IDE, obteniendo el código fuente desde el repositorio oficial para poder experimentar con modificaciones.

La prueba incluyó la extensión del modelo de metadatos, creando una nueva entidad (“dashboard2”) y un nuevo tipo de plataforma, lo que permitió validar la flexibilidad del sistema para adaptarse a necesidades específicas. También se exploró el uso de la CLI de DataHub para la gestión de entidades y la automatización de la ingestión de datos, así como la extensión del API GraphQL para que las nuevas entidades fueran accesibles en consultas avanzadas. Finalmente, se adaptó la interfaz de usuario en el frontend basado en React para mostrar la nueva entidad, evaluando la complejidad de integrar cambios visuales.

Esta primera iteración mostró que, aunque DataHub es flexible y potente, modificar y extender su modelo de datos requiere tiempo y comprensión profunda de su arquitectura, especialmente por la complejidad del sistema y la documentación incompleta en algunos casos. Los aprendizajes de esta fase servirán como base para trabajar con datos reales y automatizar la gestión de metadatos en las siguientes etapas del proyecto.

2.5 Requisitos

El sistema debe permitir la importación y representación completa de metadatos geoespaciales procedentes de ISO 19115/19139 y del modelo DCAT/GeoDCAT-AP, preservando su estructura jerárquica y semántica. Para ello, es necesario garantizar que las entidades fundamentales del modelo —Dataset, Catalog, DataService, Distribution y CatalogRecord— puedan replicarse dentro de DataHub mediante nuevas entidades, aspectos y propiedades extendidas.

Asimismo, el sistema debe ser extensible para permitir la incorporación futura de nuevos tipos de recurso o propiedades sin afectar a la compatibilidad del catálogo existente. La interfaz de usuario debe posibilitar la exploración, visualización y edición de los metadatos, mientras que el acceso programático debe estar soportado mediante API GraphQL y herramientas de ingesta automatizable.

Entre los requisitos no funcionales más relevantes se incluyen la escalabilidad —para soportar catálogos amplios y heterogéneos—, la mantenibilidad del modelo diseñado y la compatibilidad con distintos entornos de despliegue e integración con herramientas externas.

En el Anexo C se puede consultar la tabla de requisitos completa.

3 Diseño del sistema

Nuestro objetivo es recrear el esquema de metadatos DCAT dentro de nuestro entorno de DataHub. Para ello, el primer paso consistirá en analizar las entidades y estructuras que el sistema proporciona, con el fin de identificar aquellas que resulten equivalentes o compatibles con los componentes definidos en el modelo DCAT.

3.1 Arquitectura del sistema final

La arquitectura del sistema a construir tiene dos partes. Por un lado, el pipeline de obtención, transformación e ingesta de metadatos ISO que abarca desde su obtención en un endpoint CSW (para las pruebas utilizaremos el de la IDEE) hasta su ingesta por una instancia de DataHub y de otro lado la extensión de esa instancia de DataHub para su exposición en GraphQL y su visualización. Esta extensión modifica el servicio de metadatos (GMS) y la capa de acceso (GraphQL y Web UI), para soportar los nuevos tipos de contenido generados por el pipeline de procesamiento. En la figura siguiente se muestra cómo sería la arquitectura resultante (en verde los nuevos componentes desarrollados para este proyecto).

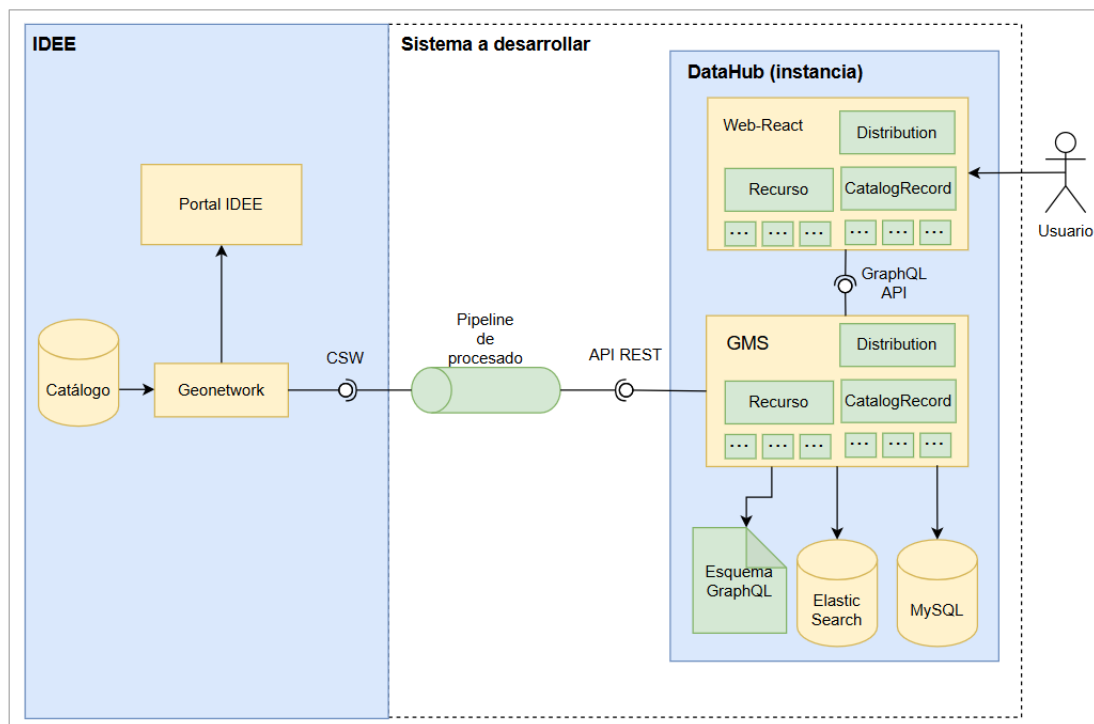


Figura 9 - Diagrama de arquitectura del sistema final

Obtención y transformación. El proceso comienza con los metadatos geoespaciales proporcionados por la IDEE en formato XML ISO 19139. Estos documentos contienen todos los elementos estructurales necesarios para reconstruir el vocabulario DCAT y GeoDCAT-AP. En un escenario real estos ficheros se recuperarían mediante un servicio CSW.

El XML es enviado a un *pipeline* de procesado, implementado en Python, responsable de:

- parsear el fichero ISO 19139,
- identificar campos relevantes,
- normalizar su contenido,
- mapearlo a las clases DCAT / GeoDCAT-AP,
- y generar objetos estructurados según el modelo extendido de DataHub.

Este pipeline produce objetos listos para su ingesta, generando la información necesaria para las entidades.

Ingesta en DataHub. Una vez transformados, los metadatos se envían al **GMS**, que es el componente central de DataHub responsable de gestionar entidades, validar aspectos, versionarlos y almacenarlos.

En este proyecto fue necesario modificar e implementar nuevas entidades dentro de GMS, añadiendo:

- los PDL (Pegasus Data Language) de las entidades,
- sus aspectos asociados,
- las conversiones de URN a los aspectos clave,
- las funciones de resolución para GraphQL que pueblan su esquema a partir de los aspectos,
- y la configuración de ingesta para cada nueva entidad.

Este punto es clave: DataHub no puede ingerir una entidad que no esté previamente definida en GMS, por lo que esta ha sido la parte central del trabajo de implementación.

Exposición del modelo: API GraphQL. Una vez almacenados en GMS, los metadatos pasan a estar disponibles a través del GraphQL API, que consulta el grafo de entidades y aspectos, resuelve relaciones entre recursos, expone atributos enriquecidos, y permite edición o actualización selectiva de aspectos.

La capa GraphQL fue extendida para incluir las nuevas entidades, resolutores y propiedades necesarias para navegar por el catálogo activo.

La selección de GraphQL es coherente con la arquitectura de DataHub, ya que el modelo interno es un grafo y GraphQL facilita consultas orientadas a relaciones.

Visualización y edición: Web-React. Finalmente, la información se refleja en la interfaz web de DataHub (frontend React), que consulta GraphQL y representa la visión de las entidades del sistema.

Para que estas entidades aparecieran en la UI fue necesario:

- registrar los nuevos tipos de entidades,
- definir sus vistas,
- configurar sus páginas de detalle, y
- habilitar la edición de aspectos seleccionados.

Esta capa constituye el punto de interacción final con el usuario, permitiendo búsquedas, navegación, edición y análisis del catálogo.

3.2 Elementos principales

El sistema resultante se estructura en torno a un conjunto de entidades, aspectos, propiedades y extensiones que permiten representar de forma completa el esquema DCAT.

Los elementos principales con sus equivalentes son los siguientes:

Tabla 1 - Entidades equivalentes DCAT-DataHub

DCAT	Datahub
Dataset	Recurso
Catalog	Recurso
DataService	Recurso
DataSetSeries	Dashboard
Resource	PlatformResource
CatalogRecord	CatalogRecord
Distribution	Distribution

3.2.1 Entidades del modelo

Dataset (reutilizada) → Recurso. DataHub dispone de una entidad Dataset cuyo modelo se ajusta casi por completo a la clase dcat:Dataset. La mayoría de los atributos esenciales —identificador, título, descripción, distribuciones, relaciones, editor, temas o etiquetas— ya se encuentran implementados mediante aspectos genéricos como:

- DatasetProperties: datos básicos del dataset (título, descripción, idioma, nombre técnico, etc.).
- Ownership: Información sobre el propietario, responsables y políticas de control.

- **BrowsePaths:** Rutas de navegación que sitúan el recurso dentro de la jerarquía del catálogo.
- **DataPlatform:** Plataforma, sistema o tecnología desde la que proviene o se gestiona el dataset.
- **Tags:** Etiquetas temáticas o palabras clave asociadas para clasificación y búsqueda.
- **Deprecation:** Estado de obsolescencia del recurso y posibles mensajes de aviso.

Dado que las clases del modelo DCAT Dataset, Catalog y DataService comparten la mayoría de los atributos, se ha definido un enfoque unificado mediante una entidad general denominada Recurso, diferenciada internamente a través de una propiedad type con valores: Dataset, Catalog y Service

Este planteamiento simplifica el mantenimiento del modelo y evita duplicación de estructuras.

Dashboard. DataHub incluye la entidad Dashboard, que agrupa un conjunto de recursos visuales (principalmente charts) y actúa como un contenedor organizado de elementos relacionados. Aunque no forma parte del modelo DCAT, su funcionamiento es conceptualmente similar a una colección o agrupación temática de datasets, por lo que se toma como referencia para representar *DatasetSeries*.

PlatformResource (extendida). La clase dcat:Resource encuentra su equivalencia más cercana en la entidad PlatformResource, ya registrada en DataHub aunque sin implementación completa. En este proyecto se ha extendido para incorporar:

- propiedades internas del archivo de metadatos original,
- estado del recurso,
- tipo de representación,
- enlace con la entidad Recurso asociada.

Nuevas entidades creadas: CatalogRecord y Distribution. Las clases CatalogRecord y Distribution, presentes en DCAT y GeoDCAT, no tienen equivalente nativo en DataHub. Por tanto, han sido implementadas como entidades independientes, con sus aspectos propios para representar elementos como:

- fechas de creación y modificación,
- formatos y enlaces,
- documentación asociada,
- tipo de distribución,
- relaciones con el recurso principal.

Estas entidades permiten reproducir las relaciones jerárquicas propias de los modelos DCAT y GeoDCAT.

3.2.2 Aspectos creados o extendidos

Para representar adecuadamente las propiedades definidas en DCAT, se incorporan nuevos aspectos y se extienden algunos existentes:

- `distributionProperties` —datos técnicos de cada distribución
- `catalogRecordProperties` — información sobre la entrada de catálogo
- `platformResourceInfo`- información relevante sobre el archivo del metadato original.
- `relations` — relaciones jerárquicas y asociativas entre recursos.

Cada aspecto se define en Pegasus PDL y se integra en el esquema GraphQL.

3.2.3 Propiedades clave incorporadas

El diseño incorpora propiedades esenciales identificadas en el análisis del vocabulario DCAT, incluyendo:

- Identificadores persistentes (registradas por las URN propias de Datahub)
- Fechas de creación, publicación y modificación
- Etiquetas y temas (`Tags` y `GlossaryTerms`)
- Editor/responsable del recurso (representada por el aspecto `Ownership`)
- Relaciones entre recursos (`dataset-service`, `catalog-dataset`) con soporte de linaje propio de Datahub.
- Información opcional espacial o temporal.

3.2.4 Esquema resultante

Como resultado, el modelo final combina entidades nativas de DataHub, extensiones y nuevas entidades diseñadas específicamente para reproducir DCAT/GeoDCAT. Las relaciones entre recursos, distribuciones y registros de catálogo forman un grafo que respeta la estructura del estándar original como se ve en la Figura 10.

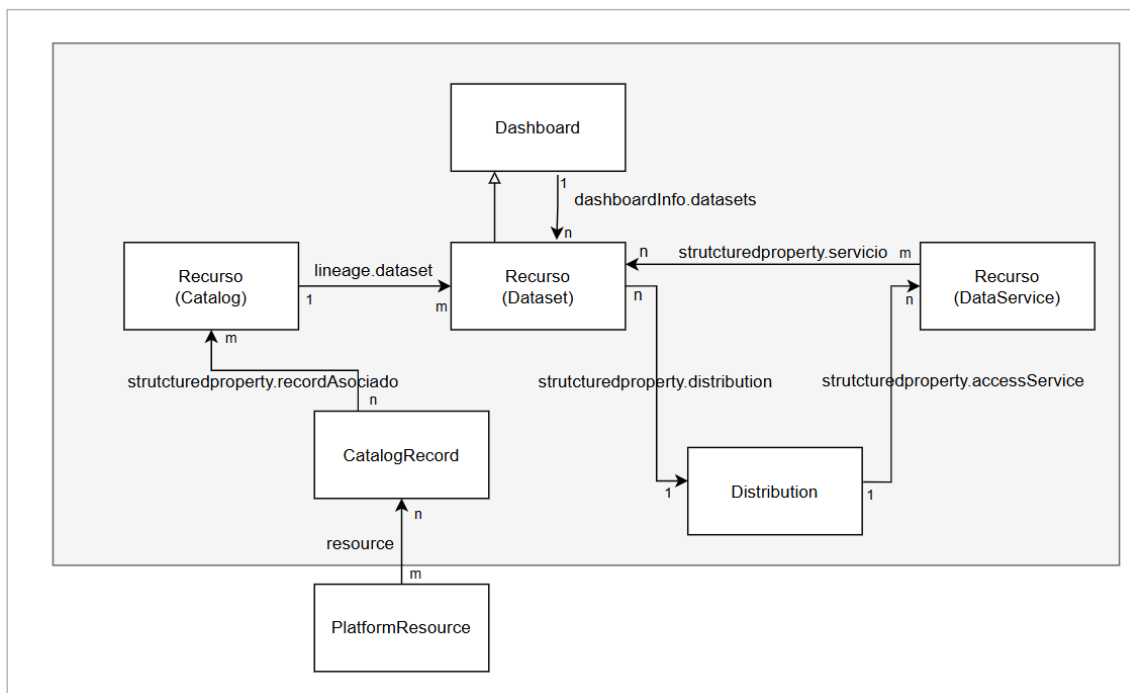


Figura 10 - Diagrama de clases resultante a la transformación

3.3 Entorno de trabajo

El entorno de trabajo ha estado compuesto por un conjunto de herramientas y tecnologías específicas para cada parte del desarrollo. La principal plataforma utilizada ha sido DataHub, desplegada en una máquina virtual Ubuntu gestionada desde un equipo Windows mediante Windows Subsystem for Linux (WSL). Para ello se utilizó Docker para la creación y gestión de contenedores que alojan la instancia del catálogo activo. Para la modificación, compilación y arranque de los ficheros Java relacionados con DataHub se utilizó el entorno de desarrollo IntelliJ IDEA, que facilitó la edición y control de versiones del código. Por otro lado, el núcleo del proyecto y las pruebas unitarias se desarrollaron en Python, empleando PyCharm[9] como entorno de desarrollo integrado, junto con el módulo pytest para la automatización y verificación de tests.

El proyecto se organizó de la siguiente manera:

- **Proyecto DataHub:** contiene la instancia de DataHub en uso, con solo los ficheros de configuración necesarios.
- **Código Java modificado:** ficheros Java dentro del proyecto los cuales se han modificado para la adición y modificación de entidades.
- **Proyecto Python:** núcleo del trabajo, que incluye la lógica principal y los tests de validación implementados con pytest.

Este entorno profesional permitió una integración fluida de los distintos lenguajes y herramientas, facilitando el desarrollo iterativo y la validación continua de la prueba de concepto.

3.4 Aspectos a destacar

El diseño del sistema ha permitido mapear de manera efectiva las entidades del modelo DCAT y GeoDCAT-AP al esquema interno de DataHub. La unificación de Dataset, Catalog y DataService bajo la entidad Recurso simplifica la gestión de metadatos y asegura una representación coherente de los recursos.

La extensión de PlatformResource y la creación de CatalogRecord y Distribution permiten conservar la información original de los metadatos, reproducir relaciones jerárquicas y cubrir propiedades que no tenían equivalencia nativa en el sistema. Los nuevos aspectos definidos garantizan que todas las propiedades esenciales estén disponibles para consultas y edición mediante GraphQL.

Estas decisiones proporcionan un modelo flexible y mantenible, listo para trasladar cada clase y propiedad del estándar DCAT al esquema de DataHub, asegurando la ingesta completa de metadatos y la preservación de su estructura jerárquica, que será el foco del siguiente capítulo.

4 Integración de DCAT en Datahub

En este capítulo se describe el proceso de adaptación y mapeo del modelo DCAT y GeoDCAT-AP al esquema interno de DataHub. Tras el análisis previo de ambos modelos, se detallan las decisiones de diseño necesarias para trasladar cada clase y propiedad, así como las extensiones incorporadas para representar correctamente los elementos que no disponen de equivalente nativo. El objetivo es definir una correspondencia completa entre los metadatos originales y su representación en DataHub, garantizando una ingesta correcta, la preservación de la estructura jerárquica y la capacidad de edición y consulta dentro del sistema.

4.1 Entidad Recurso

Para el caso de trasladar los atributos de las clases *Dataset*, *Catalog* y *DataService*, se observa que una parte considerable de sus propiedades ya está definida en DataHub y puede mapearse directamente sin necesidad de adaptaciones especiales. Este es el caso de atributos básicos como el título o la descripción, que encuentran una correspondencia directa en los aspectos nativos del sistema.

En cuanto a las keywords, DataHub ofrece dos mecanismos que permiten representarlas de forma más precisa. Las palabras clave de carácter general pueden incorporarse como *Tags*, mientras que aquellas que tienen un significado identificativo y pueden requerir una definición propia se representan como *GlossaryTerms*. Esta distinción supone una mejora respecto al esquema inicial, ya que permite reutilizar términos comunes, establecer relaciones entre recursos y facilitar su clasificación y consulta dentro del catálogo.

También resulta relevante el aspecto *Ownership*, a través del cual se representan los creadores o publicadores del metadato. DataHub permite definir tanto propietarios individuales como colectivos, lo que facilita asociar al recurso perfiles detallados de autores u organizaciones y mejorar la gestión de estos metadatos. Este sistema amplía las posibilidades de descripción y enriquece la representación de los agentes involucrados en la creación de los recursos.

Por otro lado, algunos atributos presentes en DCAT no cuentan con una correspondencia directa en DataHub, como sucede con *rights* o con elementos espaciales como el *boundingBox*. Estos pueden incorporarse sin dificultad utilizando las *StructuredProperties*, una herramienta que permite crear nuevas propiedades desde la interfaz web y asociarlas a cualquier entidad. Gracias a este mecanismo, es posible incluir todos los elementos relevantes del modelo DCAT sin pérdida de información y manteniendo la coherencia semántica del recurso.

En la siguiente tabla se muestran las propiedades principales de la clase correspondiente en DCAT y su equivalencia dentro del modelo de DataHub:

Tabla 2 - Traducción de Dataset

DCAT		Datahub	
Nombre	Tipo	Atributo	Tipo
title	string	datasetProperties.name	string
description	string	datasetProperties.description	string
keyword	string	globalTags.tags	Tag
relation	URI	datasetUpstreamLineage	Recurso
id	Identifier	structuredProperties.id	String
issued	ISO-8601 date	structuredProperties.issued	Date
modified	ISO-8601 date	structuredProperties.modified	Date
type	CodeList	structuredProperties.type	String
rights	string[]	structuredProperties.rights	string[]
language	string	structuredProperties.language	string
spatialcoverage	Extent	structuredProperties.boundingBox	string[]
publisher	string	ownership.owners[]	Owner
creator	string	ownership.owners[]	Owner
-	-	status	string
-	-	browsePaths	string[]
-	-	domains	string[]
-	-	dataPlatformInstance.platform	DataPlatformInstance

4.2 Entidad PlatformResource

Para realizar esta entidad que sería la correspondiente a *Resource* fue necesario completar su definición como se había comentado anteriormente, ya que, tal como se explicó previamente, DataHub únicamente incluía una versión parcial en su metamodelo. Tras analizar su utilidad dentro del sistema, se decidió no desarrollar su visualización en la interfaz web, tanto por la complejidad que supondría como porque la información que almacena —principalmente el fichero XML original del metadato— se procesa en fases internas de ingesta o validación mediante GraphQL. Dado que estos datos no aportan valor directo al usuario final, se optó por extender exclusivamente su representación en el esquema GraphQL.

La entidad existente en DataHub ya incorporaba el aspecto *PlatformResourceInfo*, que incluye propiedades útiles para este proyecto. Entre ellas se encuentran *resourceType* y *value*, que permiten registrar el tipo de recurso y almacenar su contenido original, así como *status*, que se utiliza para indicar si el XML asociado está completo o corresponde a una versión antigua. Estas propiedades ofrecen un punto de partida adecuado para soportar la representación básica de los recursos según el modelo DCAT.

Para otros atributos propios de DCAT que no cuentan con una correspondencia directa, se emplean de nuevo las *StructuredProperties*, que permiten añadir propiedades adicionales de forma dinámica sin modificar el núcleo del modelo. Asimismo, se pueden utilizar los campos existentes en *PlatformResourceInfo* como *primaryKey* para almacenar identificadores y *secondaryKeys* para registrar otros valores relevantes como la fecha o el lenguaje del recurso, dado que este último permite almacenar múltiples cadenas de texto.

La siguiente tabla recoge el resultado del mapeo realizado entre las propiedades definidas en DCAT para esta entidad y su correspondencia en DataHub:

Tabla 3 - Traducción de Resource

DCAT		Datahub	
Nombre	Tipo	Atributo	Tipo
title	string	datasetProperties.name	string
description	string	datasetProperties.description	string
keyword	string	globalTags.tags	Tag
id	Identifier	primaryKey	String
issued	ISO-8601 date	secondaryKey[1]	Date
modified	ISO-8601 date	secondaryKey[2]	Date
format	string	PlatformResourceInfo.ResourceType	string
language	string	secondaryKey[3]	string
publisher	string	ownership.owners[]	Owner
creator	string	ownership.owners[]	Owner
status	string	status	string
-	-	dataPlatformInstance.platform	DataPlatformInstance
content	XML	value	string

4.3 Entidad CatalogRecord

En este caso fue necesario crear una nueva entidad específica. Para ello se tomó como punto de partida la entidad Dataset, generando una copia de su definición mediante el mismo procedimiento utilizado en la prueba de concepto inicial. Al replicar esta entidad se garantiza que las relaciones con los datasets se gestionen correctamente y, además, se hereda un conjunto de atributos ya implementados y de gran utilidad, como los aspectos relacionados con *Ownership* o *Tags*, descritos en el apartado anterior.

A partir de esta estructura, se siguió la misma lógica de definición, incorporando únicamente un atributo adicional destinado a establecer la relación con la entidad *PlatformResource*. Esta relación permite acceder al recurso asociado y, en

particular, al XML original almacenado en ella, lo cual es necesario para realizar procesos de consulta, ingesta o actualización del metadato.

La tabla que se presenta a continuación recoge el resultado final del mapeo de propiedades para esta entidad:

Tabla 4 - Traducción de CatalogRecord

DCAT		Datahub	
Nombre	Tipo	Atributo	Tipo
title	string	datasetProperties.name	string
description	string	datasetProperties.description	string
source	URI	structuredProperties.source	Metadata Record
keyword	string	globalTags.tags	Tag
id	Identifier	structuredProperties.id	String
issued	ISO-8601 date	structuredProperties.issued	Date
modified	ISO-8601 date	structuredProperties.modified	Date
rights	string[]	structuredProperties.rights	string[]
language	string	structuredProperties.language	string
publisher	string	ownership.owners[]	Owner
creator	string	ownership.owners[]	Owner
-	-	browsePaths	string[]
-	-	domains	string[]

4.4 Entidad Distribution

En este caso también fue necesario crear una nueva entidad. Dado que la clase correspondiente en DCAT es más genérica y presenta un número reducido de propiedades, se optó por utilizar como referencia la entidad *Dashboard*, que es una de las más simples del modelo de DataHub. Para su creación se siguió, nuevamente, el mismo procedimiento de replicación empleado en la prueba de concepto inicial.

El aspecto clave de esta entidad reside en su relación con dos tipos de recursos: uno correspondiente a un *Dataset* y otro a un *DataService*. Ambas relaciones pueden representarse mediante *StructuredProperties*, ya que estas permiten almacenar referencias directas a otras entidades del catálogo. De este modo, se definieron dos propiedades específicas, denominadas *accessURL* y *accessService*, cada una apuntando a una entidad del tipo Recurso, según corresponda.

A continuación, se presenta la tabla con cada una de las propiedades necesarias para esta entidad junto con su correspondencia en DataHub:

Tabla 5 - Traducción de Distribution

DCAT		Datahub	
Nombre	Tipo	Atributo	Tipo
title	string	distirbutionInfo.name	string
description	string	distributionInfo.description	string
keyword	string	globalTags.tags	Tag
id	Identifier	structuredProperties.id	String
issued	ISO-8601 date	structuredProperties.issued	Date
modified	ISO-8601 date	structuredProperties.modified	Date
compressFormat	string	structuredProperties.compressFormat	String
rights	string[]	structuredProperties.rights	string[]
accessURL	Dataset	structuredProperties.accessURL	Recurso
accessService	DataService	structuredProperties.accesService	Recurso
downloadURL	string	structuredProperties.downloadURL	string

5 Pruebas de evaluación

El objetivo de esta sección es evaluar el funcionamiento del modelo implementado en DataHub mediante una serie de escenarios comparativos frente al portal de la IDEE y su representación de metadatos en formato DCAT. Con ello se pretende medir la capacidad del sistema para realizar búsquedas, mantener y actualizar metadatos, enriquecer la información disponible y comprobar su extensibilidad de forma objetiva.

Cada escenario refleja un conjunto de tareas habituales en el uso y mantenimiento de metadatos geoespaciales, y permite comparar directamente el comportamiento de ambos sistemas.

5.1 Descripción de las pruebas

Las pruebas se estructuran en varios escenarios que reproducen operaciones representativas en un catálogo de datos: búsqueda, actualización de información, enriquecimiento semántico y extensibilidad del modelo. Para cada uno de ellos se analiza cómo se comporta el portal de la IDEE, basado en metadatos DCAT simplificados, y cómo responde la implementación desarrollada en DataHub.

Aunque la plataforma de la IDEE cuenta con 23990 conjuntos de datos, no vemos necesario para este estudio cargar tanto, además del coste que requeriría. Por eso mismo, vamos a trasladar aproximadamente 1000 de estos datos para realizar las pruebas.

Los escenarios considerados son los siguientes:

- **Actualización y mantenimiento del metadato:** Comparar la capacidad de modificar, corregir o completar metadatos en ambos sistemas.
- **Recuperación de información mediante búsquedas enriquecidas:** Evaluar la diferencia entre buscar por tags, términos del glosario, autores o atributos complejos en DataHub frente al portal DCAT de la IDEE.
- **Complejidad y riqueza del metadato representado:** Analizar qué información puede representarse y consultarse en DataHub frente a la limitada visualización DCAT disponible en la IDEE.
- **Extensibilidad y escalabilidad del modelo:** Comparar la facilidad para añadir nuevas propiedades, aspectos o entidades completas en DataHub frente al modelo rígido del portal DCAT de la IDEE.

5.2 Actualización y mantenimiento de metadatos

Este escenario evalúa la capacidad de modificar o actualizar la información de un metadato existente.

En el portal de la IDEE, los metadatos publicados no pueden editarse directamente: el usuario únicamente puede visualizar la información expuesta y descargar un fichero ISO, pero no existe un mecanismo para modificar la representación DCAT ni para actualizar selectivamente sus propiedades. Cualquier cambio implica editar manualmente el XML ISO en la fuente de origen y republicarlo, lo cual dificulta las tareas de mantenimiento y validación.

En DataHub, por el contrario, la estructura basada en entidades y aspectos permite actualizar propiedades individuales (título, descripción, distribuciones, bounding box, autores...) sin necesidad de reemplazar el recurso completo. Además, mediante *StructuredProperties* es posible añadir nuevos campos sin redefinir el modelo. También en Datahub la ingesta es muy sencilla, teniendo soporte tanto para ingesta a través de scripts de Python como ingesta a través del CLI, lo que permite almacenar grandes volúmenes fácilmente. También, cuando se realiza una ingesta sobre el URN de una entidad ya existente se actualiza automáticamente con los cambios, haciendo la actualización más sencilla. Este escenario demuestra una mejora significativa en la agilidad del mantenimiento y en la calidad final del metadato.

Ejemplo concreto: modificar el autor del recurso, añadir un nuevo punto de acceso (distribution), corregir un error en el bounding box o actualizar el estado del recurso.

En IDEE solo se puede visualizar el metadato:

The screenshot displays the IDEE portal interface for a dataset. The main title is 'Unidades de demanda urbana (polígonos) PHC 2022-2027'. Below the title, there is a detailed description of the dataset, which is a WMS service. To the right of the description is a map thumbnail of Spain. Below the description, there are three sections: 'Unidades de demanda urbana (polígonos) PHC 2022-2027' with a 'Añadir al mapa' button, 'Descripción del Servicio Unidades de demanda urbana (polígonos) PHC 2022-2027' with an 'Abrir enlace' button, and 'Geoportal del Ministerio' with an 'Abrir enlace' button. On the right side, there are links for 'Provided by', 'Share on social sites', and a 'large_thumbnail' link.

Figura 11 - Captura de pantalla de un Dataset en el portal de la IDEE

Mientras que en Datahub podemos cambiar toda la información a nuestro antojo:

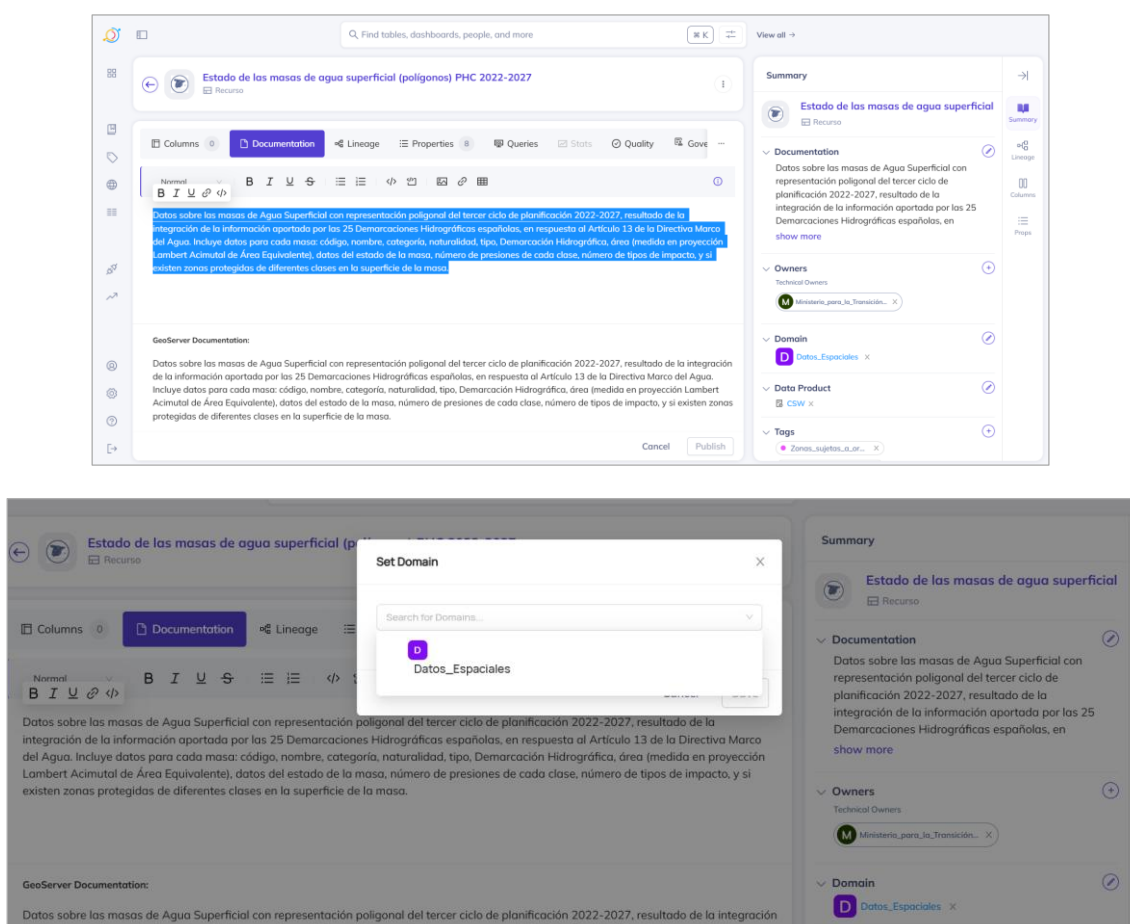


Figura 12 - Capturas de la visión y modificación de un Dataset en Datahub

5.3 Búsqueda y recuperación de información

Este escenario compara la capacidad de búsqueda entre ambos sistemas.

El portal DCAT de la IDEE únicamente permite búsquedas textuales generales, basadas en coincidencias simples sobre el título o la descripción del recurso, sin permitir filtros avanzados ni búsquedas semánticas.

En DataHub, la inclusión de tags, *GlossaryTerms*, *Ownership*, fechas, tipos de recurso y atributos espaciales permite realizar búsquedas mucho más precisas. Por ejemplo, es posible localizar recursos etiquetados con un mismo término, obtener datasets creados por una misma organización o buscar elementos que contengan una propiedad concreta generada mediante *StructuredProperties*. Además, podemos también hacer búsquedas contra la API de GraphQL, en el que con una consulta simple podemos obtener toda la información de esa entidad.

En IDEE los resultados de Búsquedas que aparecen son solo los datos que contengan la palabra concreta en título o descripción como por ejemplo si buscamos “Agua”:



Figura 13 - Captura de pantalla de la búsqueda en el portal de la IDEE

Mientras que en Datahub, podemos asignarle el Tag de “Agua” a una entidad que no tenga explícitamente en el título o descripción esta palabra, pero si tenga relación de algún tipo con esto. Se puede ver en el siguiente ejemplo que busca por el tag de “Agua” además de por palabras que aparezcan en el título o en la descripción:

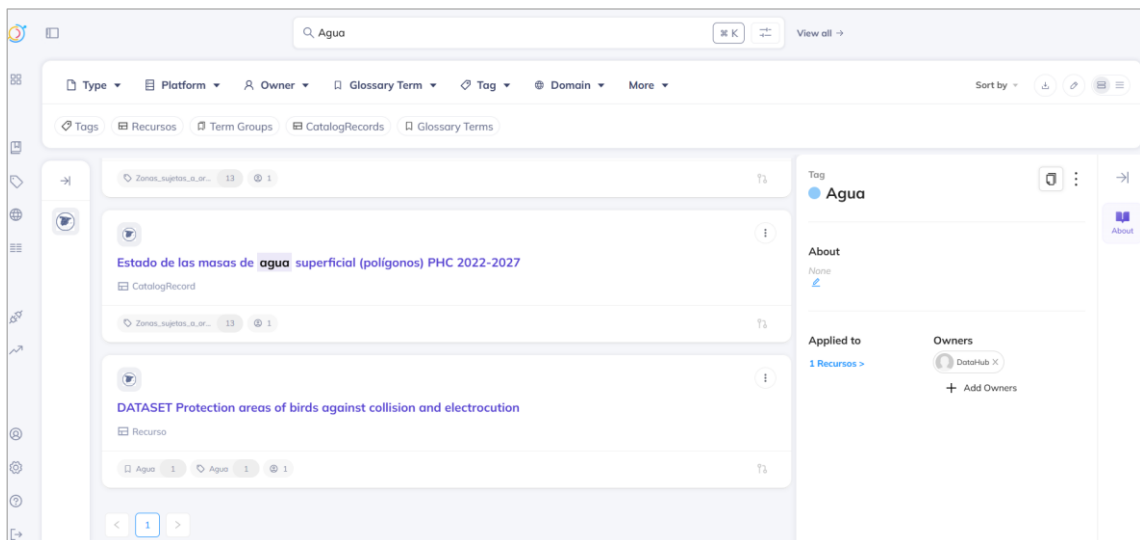


Figura 14 - Captura de pantalla de la búsqueda en Datahub

Hemos hecho que en el primero no tenga el tag de “Agua” pero en el segundo si para que el ejemplo sea concluyente:

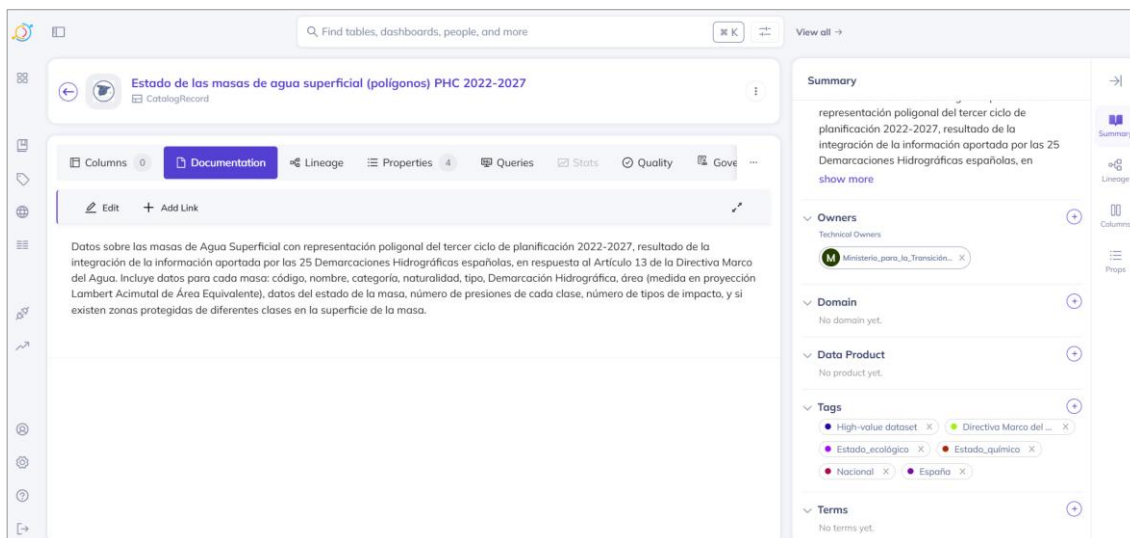


Figura 15 - Captura de pantalla de un Dataset de ejemplo con "Agua" en el título

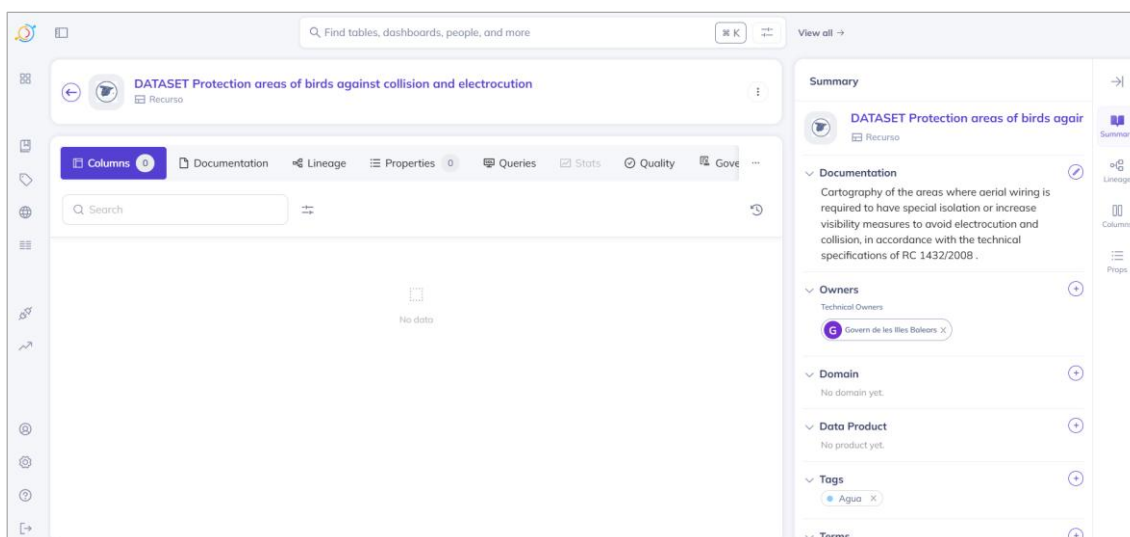


Figura 16 - Captura de pantalla de un Dataset de ejemplo con el tag "Agua"

Este escenario demuestra cómo DataHub enriquece la recuperación de información al incluir metadatos estructurados y semánticos que el modelo DCAT del portal de la IDEE no aprovecha ni expone.

5.4 Completitud y riqueza de la información mostrada

El objetivo de este escenario es comparar la cantidad y el detalle de la información accesible para el usuario.

En la IDEE, la visualización DCAT es muy limitada: ciertos elementos del metadato, como el *bounding box*, los identificadores alternativos, el quality lineage, las temporal coverages o incluso algunas distribuciones no se muestran en el portal y solo están presentes en el XML ISO original.

En DataHub, toda esta información está integrada en la entidad correspondiente gracias a los aspectos creados y a las propiedades estructuradas. Por tanto, el usuario puede consultar información técnica que no aparece en la representación DCAT simplificada de la IDEE.

Ejemplos de elementos que la IDEE no expone en DCAT pero DataHub sí:

- Bounding box exacto.
- Sistema de referencia espacial.
- Múltiples distribuciones.
- Autores individuales y organizaciones.
- Notas técnicas.
- Términos del glosario asociados.

Este escenario evidencia que DataHub no solo reproduce DCAT, sino que permite superar sus limitaciones de visualización.

5.5 Extensibilidad y escalabilidad del modelo

La IDEE publica DCAT como un modelo cerrado: las clases, propiedades y relaciones están definidas por el estándar y el portal no permite ampliarlas. Cualquier extensión debe realizarse en el XML ISO original y, aun así, no será visible ni compatible con la representación DCAT mostrada. DataHub, por su parte, permite extender el modelo mediante nuevos aspectos, entidades completas o *StructuredProperties*. Esto posibilita incorporar información adicional, adaptar el esquema a distintos dominios o integrar futuras extensiones de DCAT o GeoDCAT-AP sin modificar el núcleo del sistema. Este escenario se demuestra la escalabilidad del modelo y su capacidad para evolucionar. Como ejemplo, vemos como es la interfaz para añadir una *StructuredProperty*, que se muestra de forma bastante sencilla:

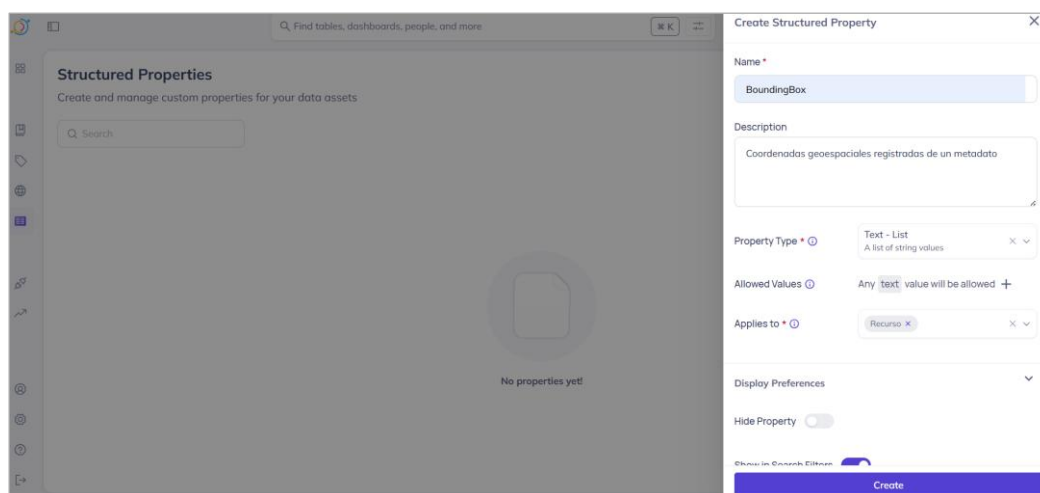


Figura 17 - Captura de pantalla de la adición de una *Structured Property*

5.6 Resultados

Las pruebas realizadas muestran que la implementación en DataHub mejora claramente la gestión de metadatos respecto al portal DCAT de la IDEE. DataHub permite actualizar metadatos de forma granular, mientras que en la IDEE la información es estática y no editable. También ofrece búsquedas más precisas gracias a etiquetas, términos de glosario y relaciones entre entidades, superando las búsquedas textuales simples del portal IDEE. Además, DataHub muestra información más completa del metadato —incluyendo elementos omitidos por la IDEE, como el bounding box o ciertas distribuciones— y su arquitectura extensible permite añadir nuevas propiedades y entidades sin modificar el modelo base. En conjunto, DataHub no solo reproduce DCAT, sino que lo mejora y lo hace más flexible y escalable para la gestión de metadatos geoespaciales.

6 Conclusiones

6.1 Objetivos conseguidos

Los objetivos planteados al inicio del proyecto se han cumplido de manera satisfactoria. Se ha logrado trasladar el modelo de metadatos definido por DCAT y sus extensiones hacia una implementación funcional dentro de DataHub, garantizando la conservación completa de la información y asegurando una solución escalable y extensible. Para ello, se han creado nuevas entidades, replicado modelos existentes, incorporado a través de nuevos aspectos y de las *StructuredProperties*, y diseñado un proceso de ingesta capaz de interpretar y almacenar correctamente los metadatos geoespaciales.

A lo largo del desarrollo se han llevado a cabo múltiples pruebas de ingesta, validación y modificación sobre todas las entidades implementadas. Estas pruebas han permitido verificar la integridad del modelo y asegurar que las operaciones de actualización, búsqueda y enriquecimiento de metadatos funcionan según lo esperado.

Aunque se han alcanzado todos los requisitos mínimos para la implementación propuesta, aún persisten ciertos puntos mejorables, especialmente relacionados con el comportamiento de algunos atributos en la interfaz web. Un ejemplo concreto es la edición de la descripción de *Distribution*, que no está incluida en las *EditableProperties* y, por tanto, no se actualiza correctamente a través de la UI, obligando a realizar la modificación mediante GraphQL. Pese a estas limitaciones menores, la versión presentada es plenamente operativa y permite trabajar con los metadatos de forma eficaz.

6.2 Limitaciones

El principal desafío del proyecto ha sido la complejidad de DataHub. Se trata de un sistema amplio, altamente modular y con numerosos componentes internos que requieren una comprensión profunda para poder extenderlo adecuadamente. La curva de aprendizaje es elevada, no solo por la estructura del modelo de datos, sino también porque DataHub integra múltiples tecnologías (Java, Kotlin, React, GraphQL, Kafka, OpenSearch, entre otras) que es necesario comprender para trabajar con fluidez.

A esta complejidad se suma que la documentación oficial no siempre está actualizada o es suficientemente detallada, lo que ha provocado retrasos y ha obligado a resolver varios problemas mediante análisis directo del código fuente y pruebas experimentales. Todas estas dificultades se han presentado

especialmente en la fase de implementación, a la hora de la extensión del modelo de GraphQL y de la implementación en el módulo web de React, ya que, aunque para la adición de entidades en GMS los errores son bastante específicos y se informan adecuadamente, por la parte de los otros módulos el proyecto muchas veces compilará correctamente, pero mostrará mal los datos en la mayoría de los casos, teniendo que buscar los fallos manualmente. Estas dificultades pueden explicar que la solución final, aun siendo sólida y funcional, pueda contener comportamientos mejorables o inconsistencias menores no críticas, como la incidencia ya mencionada en la edición del elemento *Distribution*.

En cuanto a su posible integración en otros entornos, como el portal de la IDEE, la principal limitación radica en la necesidad de adaptar el modelo de datos y la arquitectura de DataHub a las especificaciones locales. Cada instancia de DataHub está estrechamente ligada a su esquema interno, a los aspectos definidos y a las entidades personalizadas; por tanto, replicar o migrar la solución a otro portal requeriría no solo traspasar los metadatos existentes, sino también redefinir y mapear cuidadosamente todas las entidades, aspectos y relaciones para asegurar la coherencia del sistema. Este proceso de adaptación puede ser complejo y demandar un esfuerzo significativo de desarrollo y validación, especialmente si se pretende mantener compatibilidad con estándares como DCAT y GeoDCAT-AP, así como garantizar la correcta visualización y edición de los datos en la interfaz web.

6.3 Mejoras

Existen múltiples mejoras que podrían abordarse en futuras iteraciones del proyecto, especialmente si se hace el traspaso comentado en el punto anterior. En primer lugar, sería recomendable enriquecer la interfaz de DataHub para mejorar la presentación de la información de los metadatos, integrar filtros más específicos y facilitar la exploración de recursos geoespaciales. Asimismo, podría incorporarse una lógica más avanzada de validación y normalización para garantizar que todos los elementos se ingieren siguiendo estándares homogéneos.

Otro aspecto que mejorar sería la transición gradual de ciertas *StructuredProperties* hacia aspectos formales. Aunque el uso de *StructuredProperties* ha permitido agilizar y simplificar el desarrollo, definir algunos de estos atributos como aspectos nativos aportaría mayor robustez, control de versiones y reutilización dentro del modelo. Esto sería especialmente beneficioso para propiedades comunes entre entidades o elementos clave de futuras extensiones del sistema.

También podría contemplarse en la automatización del proceso de ingesta a partir de los metadatos ISO 19139 originales incorporar un sistema de validación que verifique la completitud del metadato y compruebe que todos los elementos se han

trasladado correctamente al modelo de DataHub. Además, sería útil disponer de un script o mecanismo programado que revise periódicamente la consistencia de los metadatos ya ingeridos y mantenga su información actualizada de forma automática.

Por último, si se desea, por organización y correcto versionado se podrían implementar correctamente Dataset, Catalog y DataService como entidades separadas en lugar de que estén agrupadas en la entidad Recurso diferenciándose por el tipo.

6.4 Conclusiones personales

A nivel personal, considero que el trabajo ha sido muy enriquecedor. Pese al reto que supone enfrentarse a una plataforma tan amplia como DataHub, el proceso me ha permitido adquirir una comprensión profunda sobre el funcionamiento de sistemas de catalogación de datos, el tratamiento de metadatos, la integración de modelos semánticos y el diseño de arquitecturas escalables. Además, he desarrollado habilidades de investigación, resolución de problemas y depuración que resultan fundamentales en proyectos reales de ingeniería.

Respecto a DataHub como tecnología, tras analizarla y trabajar intensivamente con ella, considero que es una alternativa bastante potente para la gestión de datos y metadatos, especialmente por su capacidad de crecimiento y por la actividad de su comunidad en GitHub. No obstante, requiere una inversión importante en tiempo y aprendizaje, y no siempre es la opción más amigable para perfiles sin experiencia previa en sistemas complejos. Con un equipo formado y un entorno organizado, DataHub puede ofrecer un marco muy sólido para gestionar información geoespacial y otros dominios a gran escala.

Bibliografía

- [1] a Rodriguez, P. Abad, and a SBnchez, “La infraestructura de datos espaciales de Espana (IDEE): un proyecto colectivo y globalizado,” *Avances en las infraestructuras de datos espaciales*, 2006.
- [2] A. Sánchez, J. Noguerras, and D. Ballari, “Normas sobre metadatos (ISO19115, ISO19115-2, ISO19139, ISO 15836),” *Mapping*, vol. 123, 2008.
- [3] A. Bhardwaj *et al.*, “DataHub: Collaborative data science & dataset version management at scale,” in *CIDR 2015 - 7th Biennial Conference on Innovative Data Systems Research*, 2015.
- [4] World Wide Web Consortium, “Data Catalog Vocabulary (DCAT),” W3C, no. January, 2014.
- [5] F. I. Maps, G. I. S. Datasets, S. Imagery, and R. Applications, “GeoNetwork - The portal to spatial data and information GeoNetwork - The portal to spatial data and information,” *Water Resources*, 2011.
- [6] w3, “W3C Web of Things,” w3.org.
- [7] F. Barthelemy *et al.*, “Towards a standard-based open data ecosystem: analysis of DCAT-AP use at national and European level,” *Electronic Government*, vol. 18, no. 2, 2022, doi: 10.1504/EG.2022.121856.
- [8] A. Perego, V. Cetl, A. Friis-Christensen, and M. Lutz, “GeoDCAT-AP: Representing geographic metadata by using the ‘DCAT application profile for data portals in Europe,’” *Joint UNECE/UNGGIM Europe Workshop on Integrating Geospatial and Statistical Standards*, 2017.
- [9] ECU-RED, *Pycharm*. 2018.

Anexo A Pasos para añadir una nueva entidad a Datahub

Para el caso de la prueba de concepto, la entidad elegida para operar ha sido “Dashboard2” la cual se ha generado tomando como referencia la entidad “Dashboard”.

A.1 Añadir un nuevo tipo de entidad

Este apartado está bien explicado en la documentación oficial.

1. Para añadir el nuevo tipo de plataforma lo primero que hay que hacer es definir su clave o “Entity Key Aspect”. En la carpeta “metadata-models/src/main/pegasus/com/linkedin/metadata/key”
2. Deberás crear un fichero pdl con el nombre de tu entidad, en nuestro caso sería Dashboard2Key.pdl y tomamos de referencia el fichero de DashboardKey.pdl para su contenido. Este fichero servirá para definir el aspecto de la entidad que quieres añadir.

```
/**
 * Key for a Dashboard2
 */
@Aspect = {
  "name": "dashboard2Key"
}
record Dashboard2Key {
  /**
   * The name of the dashboard tool such as looker, redash etc.
   */
  @Searchable = {
    "fieldName": "tool",
    "fieldType": "TEXT_PARTIAL",
    "boostScore": 4.0
  }
  dashboardTool: string

  /**
   * Unique id for the dashboard. This id should be globally unique for
   a dashboarding tool even when there are multiple> */
  dashboardId: string
}
```

3. En segundo lugar, puedes definir el fichero del aspecto con información específica sobre nuestra entidad. En la carpeta “metadata-models/src/main/pegasus/com/linkedin/” deberás crear una carpeta nueva con el nombre de tu entidad y dentro un fichero pdl con este aspecto que contendrá la información. En nuestro caso es:

```
dashboard2/dashboard2Info.pdl
```

4. Después deberás añadir la nueva entidad al fichero entity-registry.yml que se encuentra en “metadata-models/src/main/resources/”
5. Allí añadimos la sección correspondiente a nuestra entidad para que la reconozca, en nuestro caso:

```
- name: dashboard2
  keyAspect: dashboard2Key
  aspects:
    - dashboard2Info
    - dataPlatformInstance
    - browsePaths
```

6. Después se deberá reconstruir y desplegar el proyecto.
7. Para comprobar si se ha añadido correctamente deberás probar a hacer una ingesta de datos con esta entidad con el comando put, de la siguiente manera:

```
datahub put --urn
"urn:li:dashboard2:(looker,looker.com/dashboards/1234)" --aspect
dashboard2Info -d dashboard2info.json
```

8. Si se ha realizado correctamente podrás ver la URN asignada con el comando que muestra todas las URN asociadas a la entidad:

```
curl -X POST 'http://localhost:8080/entities?action=search' \
-H "Authorization: Bearer <TOKEN>" \
-H "Content-Type: application/json" \
--data '{ "input": "*", "entity": "dashboard2", "start": 0, "count":
10 }'
{"value":{"numEntities":0,"pageSize":10,"from":0,"metadata":{"aggregat
ions":[],"suggestions":[]},"entities":[]}}
```

A.2 Extender API de GraphQL

Para que nuestra nueva entidad sea reconocida en el backend deberemos extender el esquema de GraphQL y añadir esta nueva entidad.

1. Lo primero que deberemos hacer será extender el fichero `entity.graphql` que se encuentra en el directorio `“datahub-graphql-core/src/main/resources/”`

En el que deberás agregar la entidad en tres partes.

2. En

```
enum EntityType {
  ...
  DASHBOARD2
}

type Query{
  ...
  """
  Fetch a Dashboard2 by primary key (urn)
  """
  dashboard2(urn: String!): Dashboard2
  ...
}
```

3. Para que se reconozca a la entidad, y luego definirla tanto a la entidad como a su aspecto de información:

```
"""
A Dash2board Metadata Entity
"""
type Dashboard2 implements EntityWithRelationships & Entity &
BrowsableEntity {
  """
  The primary key of the Dashboard
  """
  urn: String!

  """
  A standard Entity Type
  """
  type: EntityType!

  """
  Deprecated, use properties field instead
  Additional read only information about the dashboard
  """
  info: Dashboard2Info

  """
  An id unique within the dashboard tool
  """
  dashboardId: String!

  """
  The specific instance of the data platform that this entity belongs
  to
  """
  dataPlatformInstance: DataPlatformInstance

  """
  The dashboard tool name
  """
}
```

```

    Note that this will soon be deprecated in favor of a standardized
    notion of Data Platform
    """
    tool: String!

    """
    Granular API for querying edges extending from this entity
    """
    relationships(input: RelationshipsInput!): EntityRelationshipsResult

    """
    The browse paths corresponding to the dashboard. If no Browse Paths
    have been generated before, this will be null.
    """
    browsePaths: [BrowsePath!]

    """
    Edges extending from this entity grouped by direction in the lineage
    graph
    """
    lineage(input: LineageInput!): EntityLineageResult
}

```

```

"""
Additional read only info about a Dashboard2
"""
type Dashboard2Info {
    """
    Display of the dashboard
    """
    name: String!

    """
    Description of the dashboard
    """
    description: String

    """
    Deprecated, use relationship Contains instead
    Charts that comprise the dashboard
    """
    charts: [Chart!]! @deprecated
}

```

4. Una vez añadido genera las clases necesarias con `./gradlew datahub-graphql-core:build` para que no te de ningún problema de dependencias.
5. Después, deberás implementar el tipo de la entidad en la carpeta `"datahub-graphql-core/src/main/java/com/linkedin/datahub/graphql/types"`. En esta deberás crear un directorio para tu entidad y un fichero `.java` que implemente su tipo `dashboard2/Dashboard2Type.java` Cuyo contenido lo tomaremos similar al de `DashboardType.java`. Además, puedes implementar mappers dentro de esta carpeta también.

6. Ahora deberás registrar tu entidad y sus resolvers al motor de GraphQL. Para ello, deberás modificar el fichero `GmsGraphQLEngine.java` que se encuentra en la carpeta

`“datahub-graphql-
core/src/main/java/com/linkedin/datahub/graphql/”`

7. En este añadirás los resolvers de tu entidad y su configuración, y hay que tener cuidado ya que varias funciones/secciones de este fichero deben ser modificadas:

```
private final Dashboard2Type dashboard2Type;
public GmsGraphQLEngine(final GmsGraphQLEngineArgs args) {
    ...
    this.dashboard2Type = new Dashboard2Type(entityClient);
    ...
    this.entityTypes =
        new ArrayList<>() {
            {
                ImmutableList.of(
                    ...
                    dashboard2Type,
                    ...
                )
            }
        }
    public void configureRuntimeWiring(final RuntimeWiring.Builder builder)
    {
        ...
        configureDashboard2Resolvers(builder);
        ...
    }

    private void configureQueryResolvers(final RuntimeWiring.Builder
    builder) {
        ...
        .dataFetcher("dashboard2", getResolver(dashboard2Type))
        ...
    }

    /**
     *Esta función la deberás añadir tu
     */
    private void configureDashboard2Resolvers(final RuntimeWiring.Builder
    builder) {
        //rellenar configurando tus resolvers
    }
}
```

8. Ahora, si tus resolvers son complejos, deberás definir estos resolvers que has indicado en el fichero de GMS. Para ello, deberás crear una carpeta con el nombre de tu entidad en

`datahub-graphql-
core/src/test/java/com/linkedin/datahub/graphql/resolvers/`
y definirlos.

9. Con los pasos realizados hasta ahora GraphQL ya reconoce la entidad y la genera, pero para poder operar con ella deberemos activar su búsqueda. Este paso es importante ya que en la documentación no sale bien explicado, ya que el fichero que indica ya no existe y ha sido sustituido. Para activar la búsqueda de la entidad se deberá primero modificar el fichero de SearchUtils que se encuentra en la carpeta “datahub-graphql-core/src/main/java/com/linkedin/datahub/graphql/resolvers/search/”

10. En este fichero deberemos añadirlo en tres listas:

```
public static final List<EntityType> SEARCHABLE_ENTITY_TYPES
public static final List<EntityType> AUTO_COMPLETE_ENTITY_TYPES
public static final List<EntityType> BROWSE_ENTITY_TYPES
```

11. Y si quieres que sea una de las entidades de búsqueda prioritaria la añades también en

```
public static final List<String> PRIORITIZED_SOURCE_ENTITY_TYPES
```

12. Después, un paso que no se indica en la documentación oficial y que es crucial para añadirlo a la búsqueda es que hay que añadir nuestra nueva entidad al fichero EntityTypeMapper.jsx que se encuentra en la carpeta “datahub-graphql-core/src/main/java/com/linkedin/datahub/graphql/types/entitytype”

13. Tendrás que añadirlo a la lista:

```
static final Map<EntityType, String> ENTITY_TYPE_TO_NAME =
ImmutableMap.<EntityType, String>builder()
...
    .put(EntityType.DASHBOARD2, Constants.DASHBOARD2_ENTITY_NAME)
...
```

14. También en esa misma carpeta deberás añadir la nueva entidad al fichero EntityTypeUrnMapper.jsx en el listado con las demás:

```
static final Map<String, String> ENTITY_NAME_TO_ENTITY_TYPE_URN =
ImmutableMap.<String, String>builder()
...
    .put(Constants.DASHBOARD2_ENTITY_NAME,
        "urn:li:entityType:datahub.dashboard2")
...
```

15. Y en la carpeta de “datahub-graphql-core/src/main/java/com/linkedin/datahub/graphql/types/common/mappers” deberás añadirlo al UrnToEntityMapper.java en la siguiente sección:

```
@Override
public Entity apply(@Nullable QueryContext context, Urn input) {
...
    if (input.getEntityType().equals("dashboard2")) {
```

```

partialEntity = new Dashboard2();
((Dashboard2) partialEntity).setUrn(input.toString());
((Dashboard2) partialEntity).setType(EntityType.DASHBOARD2);
}
...
}

```

A.3 Extender el UI de React

Para que la entidad aparezca en la UI de React deberemos extender los archivos de cliente de GraphQL.

1. Lo primero que debemos hacer será crear el fichero `dashboard2.graphql` similar a `dashboard.graphql` en la carpeta “`datahub-web-react/src/graphql`”
Su contenido sería algo como esto:

```

query getDashboard2($urn: String!) {
  dashboard2(urn: $urn) {
    urn
    type
    charts: relationships(input: { types: ["Contains"], direction:
OUTGOING, start: 0, count: 100 }) {
      ...fullRelationshipResults
    }
    datasets: relationships(input: { types: ["Consumes"],
direction: OUTGOING, start: 0, count: 100 }) {
      ...fullRelationshipResults
    }
    ...notes
  }
}

```

2. Después modificaremos el fichero `search.graphql` en el que se añade una entrada en los fragments de búsqueda para `Dashboard2` (hay que añadirlo en dos secciones):

```

fragment autoCompleteFields on Entity {
  urn
  type
  ...
  ... on Dashboard2 {
    info {
      name
    }
    dataPlatformInstance {
      ...dataPlatformInstanceFields
    }
  }
}

```



```

    }
    ...
  fragment searchResultsWithoutSchemaField on Entity {
    urn
    type
    ...
    ... on Dashboard2 {
      tool
      dashboardId
      info {
        name
        description
      }
      dataPlatformInstance {
        ...dataPlatformInstanceFields
      }
    }
  }
  ...

```

3. Añade la entidad en `browse.graphql` para que se pueda buscar:

```

query getBrowseResults($input: BrowseInput!) {
  browse(input: $input) {
    entities {
      urn
      type
    }
    ...
    ... on Dashboard2 {
      urn
      type
      tool
      dashboardId
      info {
        name
        description
      }
    }
  }
  ...

```

4. Implementa la interfaz `Entity` para `Dashboard2`. Para ello deberás crear una carpeta correspondiente a tu entidad en “datahub-web-react/src/app/entity” y crea su archivo `tsx` (en nuestro caso será `Dashboard2Entity.tsx`). Además, podrás crear también su `profile` o más aspectos estéticos.
5. Después deberás registrar la entidad en el `EntityRegistry`, pero en este caso los ficheros que lo gestionan han cambiado y ya no se hace en `App.tsx`, sino que está desglosado, hecho que en la documentación está desactualizado. Para este paso, deberás modificar dos ficheros: `buildEntityRegistry.ts` y `buildEntityRegistryV2.ts` en la carpeta de `/app`.
6. Una vez hayas modificado todo deberás reconstruir la UI.

Anexo B Modelo DCAT completo

Se incluye en este anexo el diagrama completo del modelo de metadatos DCAT para mostrar con precisión el punto de partida del proyecto y la complejidad real del estándar. Esta imagen permite visualizar de forma directa las entidades, propiedades y relaciones del modelo, facilitando la comprensión de cómo se han identificado los atributos relevantes para la integración en DataHub y a qué elementos del estándar corresponde cada uno.

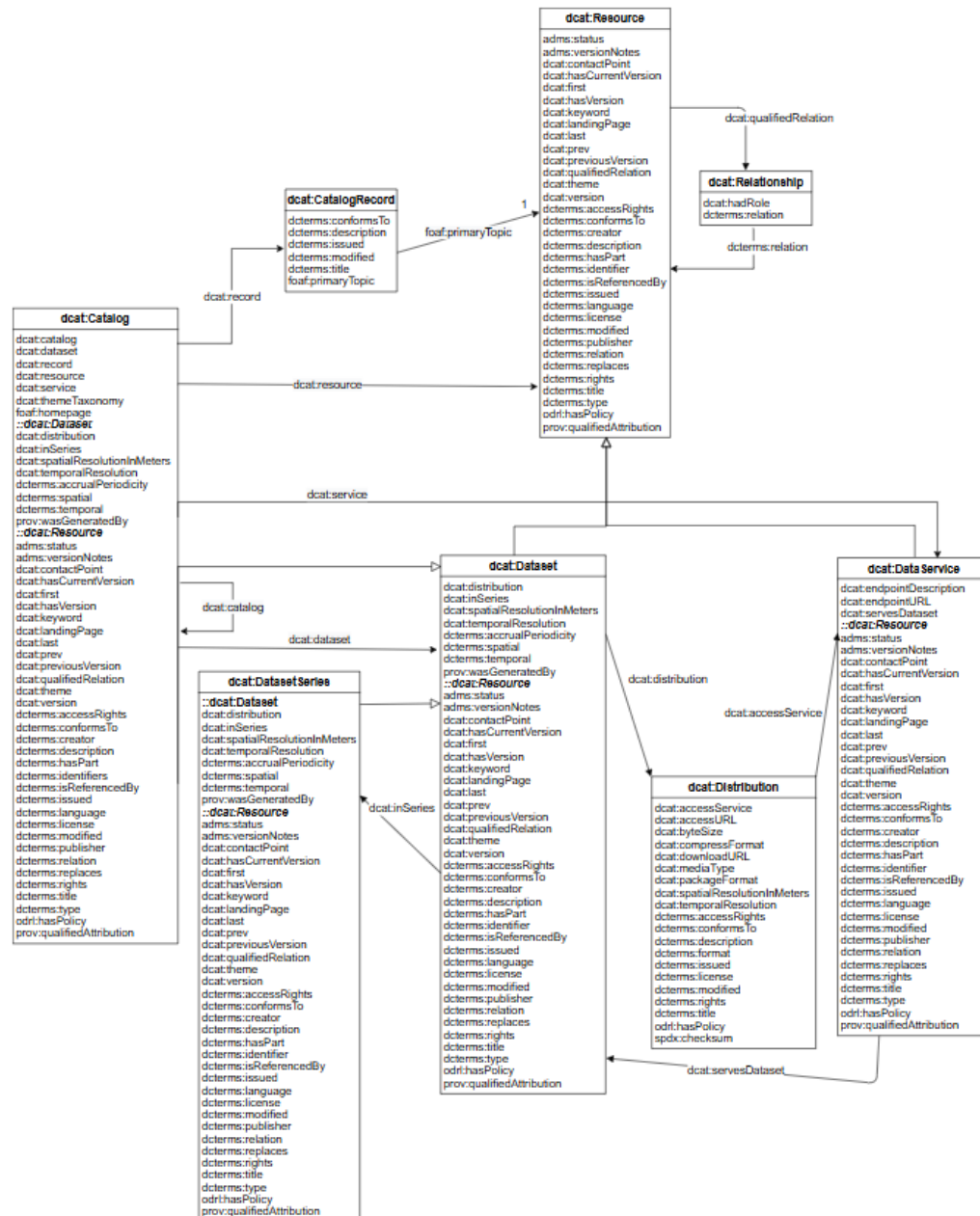


Figura 18 - Diagrama de clases de DCAT completo

Anexo C Tabla de requisitos

La siguiente tabla recopila los requisitos funcionales y no funcionales identificados para el proyecto, detallando las capacidades esenciales que el sistema debe cumplir y que sirve como referencia clara para guiar el desarrollo del proyecto.

Tabla 6 - Tabla de requisitos

Tipo de requisito	Requisito	Descripción resumida
Funcional	Importación de metadatos	Soportar ISO 19115/19139 y DCAT/GeoDCAT-AP, preservando jerarquía y consistencia.
Funcional	Representación de entidades	Incluir Catalog, Dataset, Distribution, DataService y CatalogRecord con sus relaciones completas.
Funcional	Mapeo semántico	Mantener equivalencias correctas entre clases DCAT/GeoDCAT e instancias DataHub.
Funcional	Validación	Comprobar completitud y coherencia de los metadatos ingeridos.
Funcional	Extensibilidad del modelo	Permitir nuevas entidades, aspectos y propiedades sin romper compatibilidad.
Funcional	Interfaz de usuario	Explorar, visualizar y editar metadatos; mostrar nuevas entidades y aspectos.
Funcional	Acceso programático	API GraphQL y CLI para consultas, ingestión y gestión automatizada de datos.
Funcional	Gestión de distribuciones	Representar correctamente formatos, enlaces y estructuras de distribución.
No funcional	Escalabilidad	Gestionar catálogos amplios y heterogéneos; admitir múltiples fuentes y dominios.
No funcional	Mantenibilidad	Facilitar extensiones, modificaciones y documentación sin interrumpir el catálogo.
No funcional	Compatibilidad y portabilidad	Despliegue local o en la nube; integración con herramientas externas mediante API abiertas.
No funcional	Robustez	Manejar fallos en ingestión/transformación sin perder integridad del catálogo.
No funcional	Rendimiento	Consultas y búsquedas eficientes en catálogos con muchos recursos.